

RTL Design Sherpa

Reed-Solomon Codec Micro-Architecture Specification 0.1 (draft)

October 4, 2026

Table of Contents

1 Reed Solomon Mas Index....11
2 Document Information....12
2.1 Reed-Solomon Codec Micro-Architecture Specification....12
2.2 Revision History....13
2.3 Document Purpose....14
2.4 Intended Audience....15
2.5 Related Documents....16
3 Methodology....17
3.1 HAS to MAS relationship....17
3.2 Retroactive posture: the RTL is the ground truth....18
3.3 FSM policy....19
3.4 Verification posture....20
3.5 Carrying open PRD decisions....21
4 Block Inventory....22
4.1 Drift from the pre-RTL catalog....26
5 Encoder Core (rs_encoder_core)....28
5.1 Purpose....29
5.1.1 Figure 2.1: Encoder core block diagram....29
5.2 Parameters....30
5.3 Interface....32
5.4 Microarchitecture internals....34
5.4.1 GF(2^m) LFSR parity generator....34

5.4.2 Output parity select....34
5.4.3 Output skid buffer....35
5.4.4 Frame-length check....35
5.5 FSM policy....36
5.6 Timing....37
5.7 Notes....38
6 Syndrome Unit (syndrome_unit)....39
6.1 Purpose....40
6.1.1 Figure 2.2: Syndrome unit block diagram....40
6.2 Parameters....41
6.3 Interface....42
6.4 Microarchitecture internals....43
6.4.1 Syndrome recurrence....43
6.4.2 Parallel cells....43
6.4.3 All-zero flag....43
6.4.4 Combinational next outputs....43
6.5 FSM policy....44
6.6 Timing....45
6.7 Notes....46
7 Key Equation Solver — riBM (key_equation_solver_ribm)....47
7.1 Purpose....48
7.1.1 Figure 2.3: riBM solver block diagram....48
7.2 Parameters....49
7.3 Interface....50

7.4 Microarchitecture internals..... 52
7.5 FSM policy..... 53
7.6 Timing..... 54
7.7 Notes..... 55
8 Key Equation Solver — Euclidean (key_equation_solver_euclid)..... 56
8.1 Purpose..... 57
8.1.1 Figure 2.4: Euclid solver block diagram..... 57
8.2 Parameters..... 58
8.3 Interface..... 59
8.4 Microarchitecture internals..... 61
8.5 FSM policy..... 63
8.6 Timing..... 64
8.7 Notes..... 65
9 Chien Search (chien_search)..... 66
9.1 Purpose..... 67
9.1.1 Figure 2.5: Chien search block diagram..... 67
9.2 Parameters..... 68
9.3 Interface..... 69
9.4 Microarchitecture internals..... 70
9.4.1 t + 1 Chien cells..... 70
9.4.2 Lane structure..... 70
9.4.3 Constant-multiply implementation..... 70
9.5 FSM policy..... 71
9.6 Timing..... 72

9.7 Notes..... 73
10 Forney Evaluator (forney_evaluator)..... 74
10.1 Purpose..... 75
10.1.1 Figure 2.6: Forney evaluator block diagram..... 75
10.2 Parameters..... 76
10.3 Interface..... 77
10.4 Microarchitecture internals..... 78
10.4.1 t Omega cells..... 78
10.4.2 Lane numerator..... 78
10.4.3 Per-lane inverse and multiply..... 78
10.5 FSM policy..... 80
10.6 Timing..... 81
10.7 Notes..... 82
11 Erasure Unit (rs_erasure_unit)..... 83
11.1 Purpose..... 84
11.1.1 Figure 2.7: Erasure unit block diagram..... 85
11.2 Parameters..... 86
11.3 Interface..... 87
11.4 Microarchitecture internals..... 89
11.4.1 Receive half — recording the flagged locations..... 89
11.4.2 Transform half — Gamma and the Forney syndrome..... 89
11.4.3 Solver input window..... 89
11.4.4 Combine — building the walk polynomials..... 90
11.4.5 Correction bound..... 90

11.4.6 Elaboration guard....90
11.5 FSM policy....91
11.6 Timing....92
11.7 Notes....93
12 Decoder Core Integration (rs_decoder_core)....94
12.1 Purpose....95
12.1.1 Figure 2.8: Decoder core block diagram....96
12.2 Parameters....98
12.3 Interface....100
12.4 Microarchitecture internals....102
12.4.1 Stage A — receive....102
12.4.2 Stage B — solve....102
12.4.3 Stage C — correct....102
12.4.4 Descriptor records....103
12.4.5 FIFO sizing....104
12.4.6 Status encodings....104
12.5 FSM policy....106
12.6 Timing....107
12.7 Notes....108
13 Core Signals....109
13.1 Reset strategy....110
13.2 rs_encoder_core signal contract....111
13.3 rs_decoder_core signal contract....114
13.4 Timing rules....117

14 Signal Contracts and Golden Model....118
14.1 What a contract is....118
14.2 Which signals carry the contracts....119
14.3 Citation posture....121
14.4 The golden-model contract....122
14.5 Where formal verification stands....123

List of Figures

Figure 2.1: Encoder core block diagram....29
Figure 2.2: Syndrome unit block diagram....40
Figure 2.3: riBM solver block diagram....48
Figure 2.4: Euclid solver block diagram....57
Figure 2.5: Chien search block diagram....67
Figure 2.6: Forney evaluator block diagram....75
Figure 2.7: Erasure unit block diagram....85
Figure 2.8: Decoder core block diagram....96

List of Tables

Table 1: Table 0.1: Related documents....16
Table 2: Table 1.1: MAS block inventory....22
Table 3: Table 1.2: GF leaf primitives....24
Table 4: Table 1.3: Support blocks covered elsewhere....24
Table 5: Table 1.4: Catalog-to-RTL drift....26
Table 6: Table 2.1: Encoder core parameters....30
Table 7: Table 2.2: Encoder core ports....32
Table 8: Table 2.3: Syndrome unit parameters....41
Table 9: Table 2.4: Syndrome unit ports....42
Table 10: Table 2.5: riBM solver parameters....49
Table 11: Table 2.6: riBM solver ports....50
Table 12: Table 2.7: Euclidean solver parameters....58
Table 13: Table 2.8: Euclidean solver ports....59
Table 14: Table 2.9: Chien search parameters....68
Table 15: Table 2.10: Chien search ports....69
Table 16: Table 2.11: Forney evaluator parameters....76
Table 17: Table 2.12: Forney evaluator ports....77
Table 18: Table 2.13: Erasure unit parameters....86
Table 19: Table 2.14: Erasure unit ports....87
Table 20: Table 2.15: Decoder core parameters....98
Table 21: Table 2.16: Decoder core ports....100
Table 22: Table 2.17: Descriptor record layouts (DAB_W and DBC_W)....103
Table 23: Table 2.18: Decoder core FIFO sizing....104
Table 24: Table 2.19: Decoder core status outputs....104
Table 25: Table 3.1: rs_encoder_core signal contract....111
Table 26: Table 3.2: rs_decoder_core signal contract....114
Table 27: Table 4.1: Contract areas in the landed RTL....119

List of Waveforms

No waveforms in this document.

1 Reed Solomon Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Document Information

2.1 Reed-Solomon Codec Micro-Architecture Specification

Property	Value
Document Title	Reed-Solomon Codec Micro-Architecture Specification
Version	0.3 (draft)
Date	October 4, 2026
Status	Draft — written retroactively against landed RTL (encoder and decoder cores, both solvers, gate DV green); the `.sv` files cited in this document are the ground truth
Classification	Open Source - MIT License

2.2 Revision History

Version	Date	Author	Description
0.1	2026-10-04	RTL Design Sherpa	First draft, written after the RTL landed. Documents the landed micro-architecture block by block: signal-level structure, FSM policy, cycle behavior, and the verification posture. Where the FUB catalog (`docs/rs_fub_catalog.md`, 2026-09-29) and the RTL disagree, the RTL wins and the catalog is noted as the plan this MAS checks.
0.2	2026-10-20	RTL Design Sherpa	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.
0.3	2026-10-20	RTL Design Sherpa	Section (h2) headings now open a new page, the pipeline default. No technical content changed.

2.3 Document Purpose

This Micro-Architecture Specification (MAS) is the implementation-level view of the Reed-Solomon codec — the HAS (`../reed_solomon_has/`) taken one level down. It is the document a verification engineer reads before writing assertions and the document an integrator reads before instantiating a core.

It covers:

- each functional block's internal structure, signals, and parameters, cited to the `.sv` source
- the FSM policy for every block: where a state machine is forbidden, where one minimal control FSM is permitted, and what replaces FSMs elsewhere
- cycle-level timing and measured throughput per block
- the verification posture: golden-model equivalence, the dual-solver cross-check, and the contract discipline

2.4 Intended Audience

- Verification engineers extending the cocotb gate suite or writing formal properties
- Integrators wiring the cores or the standalone tops into a consumer
- Architects weighing a KES_ALGO switch, an erasure enable, or an S > 1 throughput profile

2.5 Related Documents

Table 0.1: Related documents

Document	Location	Content
Product Requirements	`../../PRD.md`	the decision table (D1-D12) and the candidate profiles
Hardware Architecture Specification	`../reed_solomon_has/reed_solomon_has_index.md`	high-level block diagram, data flow, solver options, interfaces, parameters
FUB catalog	`../../rs_fub_catalog.md`	the bottom-up build plan this MAS checks the landed tree against
Golden model	`../../../dv/tbclasses/rs_model.py`	the Python reference, itself validated against `reedsolo`/`galois`
References	`../../References/README.md`	the standards and papers, with source and licence
CLAUDE.md	`../../../CLAUDE.md`	area facts for a session working here
Handbook	`vault/handbook/design/`	streaming-no-fsm, minimal-fsm, signal-contracts-and-kmaps, valid-ready contracts

3 Methodology

3.1 HAS to MAS relationship

The Hardware Architecture Specification (`../reed_solomon_has/`) records what the RS codec is, where it sits, what it presents at its boundaries, and what parameters configure it. This Micro-Architecture Specification carries that architecture one level down: per-block signal tables, cycle behavior, FSM policy, and the verification references a checker needs.

The rule in this repo is one source per fact. The MAS does not restate the PRD decision table or the HAS block diagram; it cross-references them and adds the implementation detail that belongs at this level. Parameters that remain open are marked **TBD** and tied to the PRD decision ID that fixes them, following the convention in `../reed_solomon_has/ch06_integration/02_parameters.md`.

3.2 Retroactive posture: the RTL is the ground truth

The BCH MAS was written before its RTL existed; its signal contracts cite MAS pages with the promise to re-point them at `.sv` lines later. This MAS is written retroactively, after the RS RTL landed and its gate DV went green, so it takes the other posture from the start:

- every block page cites the `.sv` file it documents, and expressions in the text mirror the RTL rather than an intent the RTL must catch up to
- where the pre-RTL FUB catalog (`docs/rs_fub_catalog.md`) predicted a different decomposition — it planned an `erasure_locator` leaf, a `symbol_unpack`/`symbol_pack` pair, and a separate `parity_mux`; the landed tree routes those jobs differently — the block page says what actually happened and the inventory (chapter 1.2) collects the drift
- nothing here invents a numeric value; measured cycle counts come from the gate logs cited on the block pages

3.3 FSM policy

The repo's datapath FSM rule, taken from the design handbook, is:

> **No FSM in the per-beat data path. Not a minimal one, not a two-state one.** FSMs belong to control: descriptor lifecycle, schedulers, init sequencers, error recovery. If a state machine can observe a data beat, it is in the wrong place. — `vault/handbook/design/streaming-no-fsm.md`

What replaces a datapath FSM in this block:

Instead of a state that…	Write
counts beats, or waits N cycles	a counter plus a comparator
remembers "I am mid-burst"	a `r_valid` flag on the pipeline stage
serializes because a resource is shared	an arbiter and a qualifier, not a turn-taking machine
gates until a condition holds	a qualifier ANDed into the handshake

Where a state machine is genuinely justified, the handbook requires:

> Fewest states that carry the real distinctions. A state that only waits one fixed cycle, or that differs from a sibling only by a datapath value, is a register, not a state. Merge states whose outputs and successors are identical. Idiom: `typedef enum logic [N-1:0] {...} state_t;` + two-process form - registered `r_state`, combinational `w_next_state` with a default-hold first line and a default arm. One FSM per module; nested/communicating FSMs are a smell that the block wants splitting. Outputs: prefer registered (Moore) at module boundaries; combinational decode of `r_state` internally is fine but belongs in the K-maps. — `vault/handbook/design/minimal-fsm.md`

Every block page in chapter 2 states its FSM policy explicitly and cites this page. Most blocks are pure datapath and carry no FSM; the control FSMs that do exist — the Euclid solver's iterate/normalise/swap loop and the standalone tops' job engines — are documented with their state enumerations.

3.4 Verification posture

Three layers, all live today:

1. **Golden-model equivalence.** Every block is bit-exact against `dv/tbclasses/rs_model.py`, which is itself validated against the `reedsolo` (and `galois`) packages — never a hand-rolled GF table in the test.
2. **Dual-solver cross-check.** `KES_ALGO = "RIBM"` (default) or `"EUCLID"`; both solvers decode the same injected stream and must produce identical Chien root sets and identical Forney values for every corrected block (`error_injector` feeds both, per `CLAUDE.md`).
3. **Contract discipline.** The repo's machine-checkable contract methodology (`vault/handbook/design/signal-contracts-and-kmaps.md`) has no generated workbook for RS yet; chapter 4 states what the contracts are, where they live in the RTL, and what generating a workbook would take. The BCH component's generator (`projects/components/ecc-ip/bch/docs/gen_bch_signal_contracts_kmaps.py`) is the pattern.

3.5 Carrying open PRD decisions

Every undecided item in the PRD is carried as **TBD** with the decision ID that resolves it. As of this revision:

Parameter / choice	PRD decision	Where it lands in the RTL
erasure support (`ENABLE_ERASURES`)	D5	`rs_erasure_unit`, generated only when enabled; default 0 leaves it out of the tree
first root / dual basis (`FIRST_ROOT`, `DUAL_BASIS`)	D8	`gf_pkg` generator construction; defaults b = 0, no dual basis
encoder-only vs both cores	D4	both cores built; a consumer instantiates what it needs
first consumer	D10	none of the above changes until one appears

Everything else — m, t, n (D1-D3), throughput architecture (D6), interface mix (D9), the solver default (D11), the scrambler (D12) — is decided and stated as fact on the block pages.

No numeric value is invented. Cycle-count formulas are stated in terms of the parameters (for example, `n/S` cycles to absorb a block), and measured numbers cite the gate log that produced them.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 Block Inventory

The MAS documents the landed blocks below. Each page follows the FUB-page skeleton from the pumice and BCH MASes: header metadata, purpose, parameters, interface, microarchitecture internals, FSM policy, timing, and notes. The landed RTL is the ground truth; the pre-RTL FUB catalog is the plan this inventory checks it against (drift collected in Table 1.3).

Table 1.1: MAS block inventory

Block	RTL	Role	MAS page
`rs_encoder_core`	`rtl/macro/rs_encoder_core.sv`	Systematic encoder core, valid/ready at both ends: data beats pass through, `gf_lfsr_encoder` computes parity, parity drains behind the data	01_encoder.md
`syndrome_unit`	`rtl/fub/syndrome_unit.sv`	The 2t syndromes `S_i = r(alpha^(b+i))` computed by `gf_syndrome_cell` Horner lanes as the block streams in; all-zero flag	02_syndrome_unit.md
`key_equation_solver_ribm`	`rtl/fub/key_equation_solver_ribm.sv`	Default solver (D11): 3t+1 `ribm_pe` cells, 2t iterations, locator + evaluator out	03_key_equation_solver_ribm.md
`key_equation_solver_euclid`	`rtl/fub/key_equation_solver_euclid.sv`	Alternative solver (D11): inversionless modified Euclidean, top-aligned	04_key_equation_solver_euclid.md

Block	RTL	Role	MAS page
		polynomials, early stop	
chien_search	rtl/fub/chien_search.sv	Locator root search over the n positions, S lanes per cycle; root flags and the odd-index derivative sum for Forney	05_chien_search.md
forney_evaluator	rtl/fub/forney_evaluator.sv	Error magnitude per lane: Omega and the derivative at each located root, one inverse and multiply per lane	06_forney_evaluator.md
rs_erasure_unit	rtl/fub/rs_erasure_unit.sv	Erasure support (D5): builds the erasure locator during receive and combines it with the syndrome locator for the solver; generated only when ERASURE_SUPPORT = 1	07_erasure_unit.md
rs_decoder_core	rtl/macro/rs_decoder_core.sv	Decoder integration: block buffer, descriptor skids, solve bypass, Chien/Forney correct walk, second syndrome	08_decoder_core.md

Block	RTL	Role	MAS page
		unit, verdict and release	

The GF(2^m) leaf primitives are documented inside their parent pages rather than given pages of their own; they are collected here so the inventory is complete:

Table 1.2: GF leaf primitives

Primitive	RTL	Used by
`gf_pkg`	`rtl/fub/gf/gf_pkg.sv`	all blocks (constant functions: multiply, powers, inverse, log, primitivity check)
`gf_mul`	`rtl/fub/gf/gf_mul.sv`	riBM/Euclid solver arrays, Forney
`gf_mul_const`	`rtl/fub/gf/gf_mul_const.sv`	syndrome cells, encoder taps, Chien load/step, Forney constants
`gf_inv`	`rtl/fub/gf/gf_inv.sv`	Forney (log/negate/antilog tables; Itoh-Tsujii above m = 12)
`gf_syndrome_cell`	`rtl/fub/gf/gf_syndrome_cell.sv`	`syndrome_unit` (2t instances)
`gf_lfsr_encoder`	`rtl/fub/gf/gf_lfsr_encoder.sv`	`rs_encoder_core`
`ribm_pe`	`rtl/fub/gf/ribm_pe.sv`	`key_equation_solver_ribm` (3t+1 instances)

Support blocks that are not taken to the signal level in this MAS:

Table 1.3: Support blocks covered elsewhere

Block	RTL	Why it has no page here
`rs_beat_packer`	`rtl/fub/rs_beat_packer.sv`	Adapter-side repacker; the HAS (chapter 4.2/4.3) specifies its job, its header documents its own contract

Block	RTL	Why it has no page here
`error_injector`	`projects/components/utility-ip/misc/rtl/error_injector.sv`	DV stimulus (shared with BCH), documented in its header; not part of the deliverable datapath
`rs_pkg`	`rtl/fub/rs_pkg.sv`	shared types/parameters package
AXIS/AXI4 tops and engines	`rtl/top/*.sv`	specified at the interface level in `../reed_solomon_has/ch04_interfaces/`

4.1 Drift from the pre-RTL catalog

The FUB catalog (`docs/rs_fub_catalog.md`, 2026-09-29) was the build plan. The landed tree decomposed a few planned blocks differently:

Table 1.4: Catalog-to-RTL drift

Catalog plan	Landed reality	Where
`erasure_locator` leaf building the erasure polynomial	`rs_erasure_unit`: erasure locator and solver combination in one generated block riding the A→B descriptor	chapter 2.7
`symbol_unpack` / `symbol_pack` / `parity_mux` leaves	beat slicing, keep handling, and the parity append are inline in the two cores; contiguous repacking is the separate `rs_beat_packer` at the adapter boundary	chapters 2.1, 2.8
`chien_cell` leaf, `euclid_pe` as generate-loop logic	same structure as planned: per-coefficient cells live inside `chien_search` and `key_equation_solver_euclid` as generate loops	chapters 2.5, 2.4
`line_randomizer` scrambler (D12, decided)	not landed yet; no scrambler RTL exists in the tree as of this revision	—

Where a block page and the catalog disagree on internals, the page wins — it was written against the `.sv` file.

5 Encoder Core (rs_encoder_core)

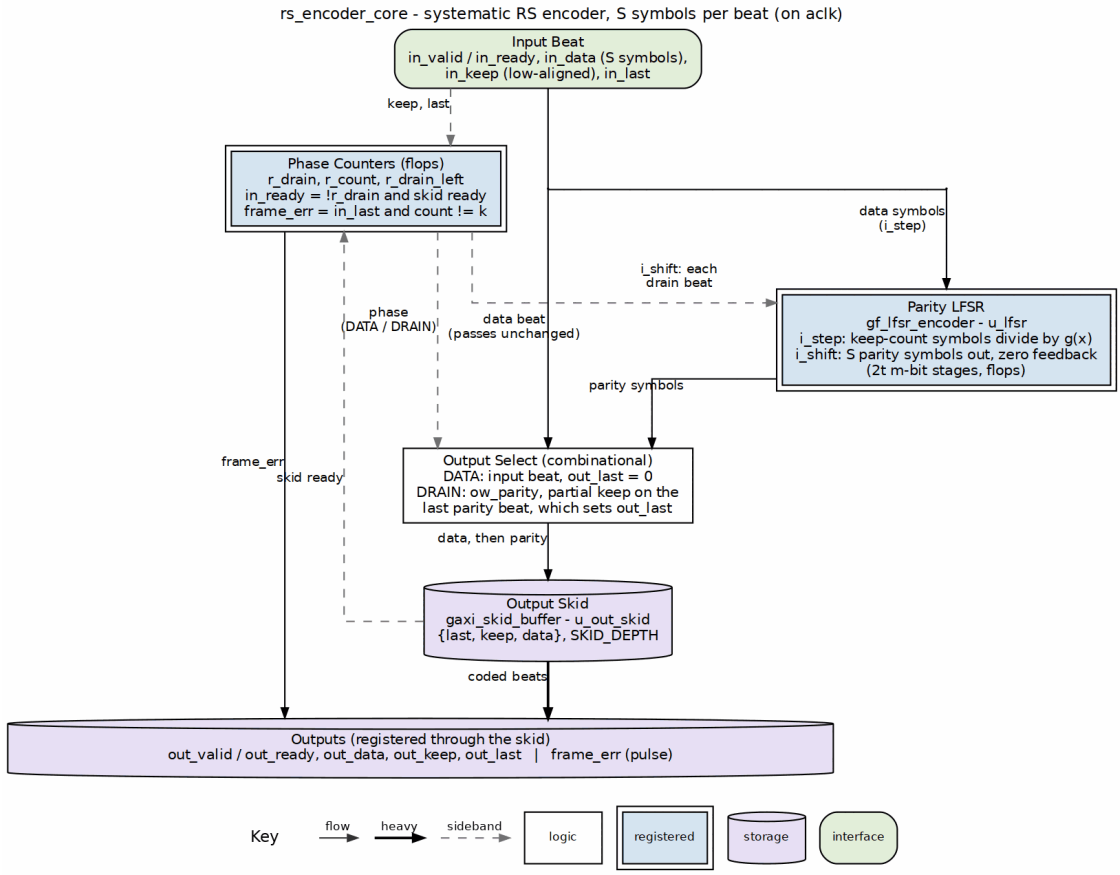
Module: rs_encoder_core.sv **Location:** rtl/macro/ **Category:** streaming datapath **Parent:** rs_encoder / standalone encoder wrapper **Status:** landed — rtl/macro/rs_encoder_core.sv, gate DV green on the standing profiles

---

5.1 Purpose

`rs_encoder_core` systematically encodes a `K_SYMBOLS`-symbol data block into an `N_SYMBOLS`-symbol Reed-Solomon codeword. Data symbols pass through unchanged; the block then appends the `2 * T_SYMBOLS` parity symbols produced by a GF(2^m) LFSR whose taps come from the generator polynomial computed by `gf_pkg`. The output stream is longer than the input stream, so the encoder is not a pure back-to-back pipeline: it holds `in_ready` low while parity drains.

5.1.1 Figure 2.1: Encoder core block diagram



Encoder core block diagram

5.2 Parameters

Table 2.1: Encoder core parameters

Parameter	Type	Range	Default	Meaning	PRD
SYMBOL_WIDTH	int	2..GF_MAX_M	8	m; field is GF(2^m)	D1
PRIM_POLY	int	primitive of degree m	0x11D	primitive polynomial; selects the field representation	D8 per profile
T_SYMBOLS	int	1..(2^m-1)/2	8	correctable symbols per block; 2t parity symbols	D2
N_SYMBOLS	int	2t+1 .. 2^m-1	2^m - 1	codeword length in symbols; < 2^m-1 is a shortened code	D3
FIRST_ROOT	int	0 .. 2^m-2	0	b, first root of g(x)	D8
DATA_WIDTH	int	multiple of SYMBOL_WIDTH	SYMBOL_WIDTH	beat width	D9
SKID_DEPTH	int	2..8	2	output skid-buffer depth	D9
K_SYMBOLS	derived	N_SYMBOLS - 2*T_SYMBOLS	—	data symbols per block	D3
SYMBOLS_PER_BEAT	derived	DATA_WIDTH / SYMBOL_WID	1	symbols per	D6

Parameter	Type	Range	Default	Meaning	PRD
		TH		valid/ready beat	

K_SYMBOLS and SYMBOLS_PER_BEAT are derived but exposed in the module header for the consumer's convenience. Elaboration fails if DATA_WIDTH is not a multiple of SYMBOL_WIDTH, if N_SYMBOLS does not exceed 2*T_SYMBOLS, or if SKID_DEPTH is outside 2..8.

5.3 Interface

Table 2.2: Encoder core ports

Signal	Direction	Width	Description
`aclk`	in	1	one clock for the whole core
`aresetn`	in	1	active-low synchronous reset
`in_valid`	in	1	a beat of data symbols is offered
`in_ready`	out	1	the core takes it this cycle
`in_data`	in	`DATA_WIDTH`	data symbols, symbol 0 in the low lane
`in_keep`	in	`SYMBOLS_PER_BEAT`	present-symbol mask, low-aligned; partial only on the block's final beat
`in_last`	in	1	this beat carries the `K_SYMBOLS`-th data symbol
`out_valid`	out	1	a beat of coded symbols is offered
`out_ready`	in	1	the consumer takes it
`out_data`	out	`DATA_WIDTH`	data symbols while `!r_drain`, parity symbols while `r_drain`
`out_keep`	out	`SYMBOLS_PER_BEAT`	present-symbol mask; partial at the end of data and at the end of

Signal	Direction	Width	Description
			parity
`out_last`	out	1	this beat carries the `N_SYMBOLS`-th coded symbol
`frame_err`	out	1	one-cycle pulse: a block ended with other than `K_SYMBOLS` data symbols

5.4 Microarchitecture internals

5.4.1 GF(2^m) LFSR parity generator

The parity generator is `gf_lfsr_encoder` in `projects/components/ecc-ip/reed-solomon/rtl/fub/gf/gf_lfsr_encoder.sv`. It is a `2 * T_SYMBOLS`-stage shift register over GF(2^m) whose tap coefficients are the generator polynomial `g(x)` built at elaboration from `gf_pkg`:

```
g(x) = prod_{i=0}^{2t-1} (x - alpha^(b+i))
```

The LFSR instance receives:

- `i_step`: high on every accepted input beat (`w_in_fire`)
- `i_data`: the input beat's `S` symbols
- `i_count`: how many symbols are present in the beat (`1 .. S`)
- `i_shift`: high on every drain beat (`w_drain_fire`)

One symbol step is polynomial division by `g(x)`:

```
fb      = data ^ r[2t-1]
r[j]    = r[j-1] ^ fb * g[j]    (j = 1 .. 2t-1)
r[0]    = fb * g[0]
```

The core unrolls `S` single-symbol steps and selects the state after `i_count` of them, so a partial final beat costs only a mux. The constant tap multiplies fold to XOR networks; the unrolled chain is one XOR network per register whatever `S` is.

5.4.2 Output parity select

The output mux selects data beats while `r_drain` is low and parity beats once parity starts draining:

```
in_ready        = !r_drain && w_skid_wr_ready
w_in_fire       = in_valid && in_ready
w_drain_fire    = r_drain && w_skid_wr_ready
```

While draining, the next `S` parity symbols come from the LFSR register in transmission order:

```
ow_parity lane u = r_reg[2t-1-u]   (u = 0 .. S-1)
```

The final parity beat is partial when `S` does not divide `2t`. `w_par_keep` marks the valid lanes:

```
w_par_keep[u] = (r_drain_left != 1) || (u < PREM)
```

where `PREM = 2t - (PB-1)*S` is the number of symbols in the last parity beat and `PB = ceil(2t / S)` is the total parity beats.

5.4.3 Output skid buffer

A `gaxi_skid_buffer` (`projects/components/ecc-ip/reed-solomon/rtl/fub/` does not own it; the block is in `rtl/amba/gaxi`) sits on the output. It decouples the consumer's `out_ready` from the LFSR step, so the encoder can take one input symbol per cycle when the consumer keeps up, and the `ceil(2t/S)`-cycle parity gap is the only stall it introduces.

5.4.4 Frame-length check

`r_count` tracks how many data symbols have been accepted in the current block. On `in_last` the core compares the expected `K_SYMBOLS` against the actual count plus the last beat's symbol count:

```
frame_err = in_last && (w_count_next != K_SYMBOLS)
```

A block that is shorter or longer than `K_SYMBOLS` still gets encoded as given (systematic data pass-through with parity appended to whatever data arrived); `frame_err` is just the one-cycle pulse that tells the consumer the block was mis-framed.

5.5 FSM policy

This block carries **no FSM** (per `vault/handbook/design/streaming-no-fsm.md`). The sequencing that another design might put in a state machine is replaced by:

- `r_drain`: registered flag, 0 in the DATA phase, 1 in the parity DRAIN phase
- `r_count`: counts data symbols accepted in the block, 0 .. `N_SYMBOLS-1`
- `r_drain_left`: counts parity beats still to emit, `PB` down to 0
- `w_in_count`: combinational count of valid symbols in the current input beat
- `w_skid_wr_valid` / `w_skid_wr_ready`: output skid handshake
- `w_in_fire` / `w_drain_fire`: combinational qualifiers for LFSR step and shift

`in_ready` is combinational `!r_drain && w_skid_wr_ready`.

5.6 Timing

- One input beat is accepted per cycle while `in_ready` is high.
- The data phase is `ceil(K_SYMBOLS / SYMBOLS_PER_BEAT)` beats.
- The parity phase is `ceil(2*T_SYMBOLS / SYMBOLS_PER_BEAT)` beats.
- `in_ready` is held low during the parity phase; back-to-back blocks need an upstream FIFO sized for at least `2*T_SYMBOLS` symbols.
- Measured: one beat per cycle plus the `ceil(2t/S)` parity gap.

5.7 Notes

- **Wrong-length block handling:** If `in_last` arrives before or after the `K_SYMBOLS`-th data symbol, `frame_err` pulses and the block is still encoded as given. This is the encoder-side analog of the decoder's framing-error status (HAS chapter 3.2).
- **Partial beats:** The encoder passes data beats through as received, so a partial data beat can occur at the end of the data phase. It then appends parity in `PB` beats, the last of which is partial when `S` does not divide `2t`. A consumer that needs contiguous packing puts a beat packer at the outlet (HAS chapter 4.2 / 4.3).
- **No separate LFSR clear:** After the last parity beat the LFSR has shifted itself to all zeros, so the next block starts clean with no explicit reset cycle.

6 Syndrome Unit (`syndrome_unit`)

Module: `syndrome_unit.sv` **Location:** `rtl/fub/` **Category:** streaming datapath **Parent:** `rs_decoder_core` **Status:** landed — `rtl/fub/syndrome_unit.sv`, gate DV green on the standing profiles

---

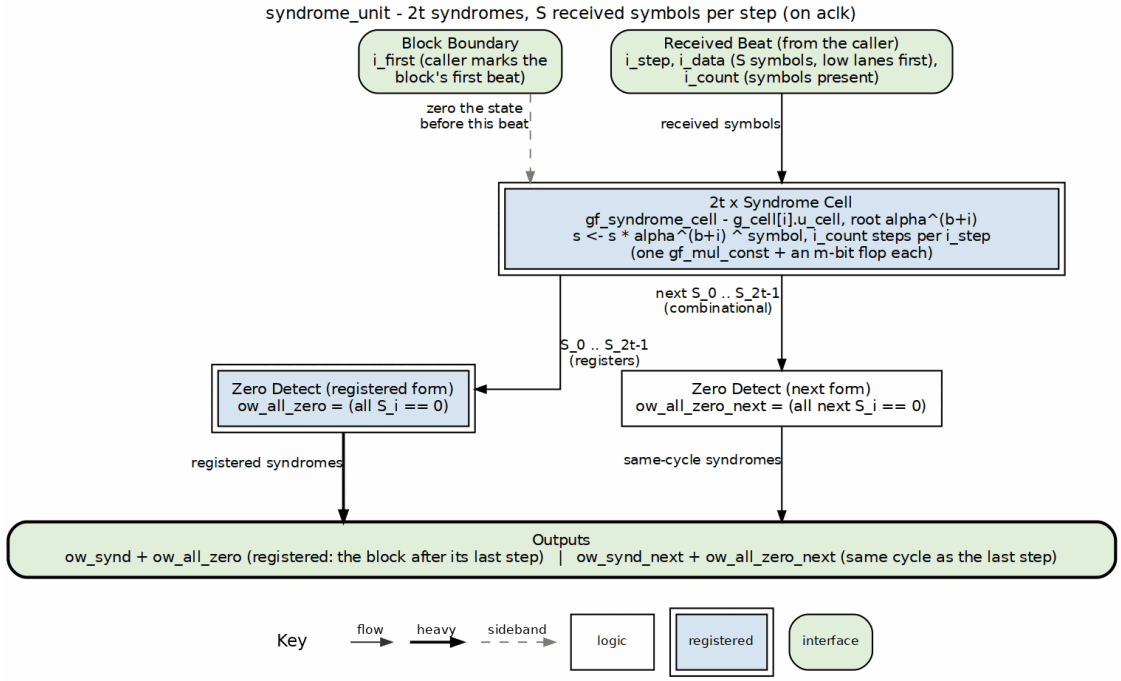
6.1 Purpose

`syndrome_unit` computes the `2t` syndromes `S_0 .. S_2t-1` of a received Reed-Solomon codeword as the symbols arrive. Syndrome `S_i` is the received polynomial evaluated at `alpha^(b+i)`:

```
S_i = r(alpha^(b+i)) = sum_{j=0}^{n-1} r_j * alpha^((b+i)*j)
```

The unit contains `2t` parallel GF(2^m) multiply-accumulate lanes, one per syndrome. It also supplies combinational `ow_*_next` versions of the outputs so a caller can capture the finished syndromes in the same cycle as the last symbol.

6.1.1 Figure 2.2: Syndrome unit block diagram



Syndrome unit block diagram

6.2 Parameters

Table 2.3: Syndrome unit parameters

Parameter	Type	Range	Default	Meaning	PRD
SYMBOL_WIDTH	int	2..GF_MAX_M	8	m; field is GF(2^m)	D1
PRIM_POLY	int	primitive of degree m	0x11D	primitive polynomial; selects the field representation	D8 per profile
T_SYMBOLS	int	1..(2^m-1)/2	8	correctable symbols per block; 2t syndromes	D2
FIRST_ROOT	int	0 .. 2^m-2	0	b, exponent of the first root alpha^b	D8
SYMBOLS_PER_BEAT	int	>= 1	1	S, symbols per i_step	D6

6.3 Interface

Table 2.4: Syndrome unit ports

Signal	Direction	Width	Description
`aclk`	in	1	one clock
`aresetn`	in	1	active-low synchronous reset
`i_step`	in	1	advance every lane by `i_count` symbols
`i_first`	in	1	zero the accumulators first (block's first beat)
`i_data`	in	`SYMBOLS_PER_BEAT * SYMBOL_WIDTH`	received symbols, low lanes first
`i_count`	in	`$clog2(SYMBOLS_PER_BEAT+1)`	valid symbols in this beat, `1 .. S`
`ow_synd`	out	`2*T_SYMBOLS*SYMBOL_WIDTH`	packed syndromes, `S_0` in the low `m` bits
`ow_all_zero`	out	1	all `2t` syndromes are zero
`ow_synd_next`	out	`2*T_SYMBOLS*SYMBOL_WIDTH`	combinational next value of `ow_synd`
`ow_all_zero_next`	out	1	combinational next value of `ow_all_zero`

6.4 Microarchitecture internals

6.4.1 Syndrome recurrence

Each lane accumulates by Horner's rule as symbols arrive in transmission order:

```
S_i <- S_i * alpha^(b+i) + r_j
```

The multiplier by the constant `alpha^(b+i)` is a `gf_mul_fn` call on a constant operand; synthesis folds it to a fixed XOR network. No per-position power table is stored; the constant root is wired once per lane.

With `SYMBOLS_PER_BEAT = S > 1`, each lane performs `S` Horner steps per cycle:

```
S_i <- (((S_i * alpha^(b+i)) + r_j0) * alpha^(b+i) + r_j1) *
alpha^(b+i) + ... + r_jS-1
```

This is a short feed-forward chain of `S` constant multiplies and adds. The state after `i_count` steps is selected by a mux, so a partial final beat costs nothing extra in area or timing.

6.4.2 Parallel cells

`syndrome_unit` instantiates `2t` copies of `gf_syndrome_cell` from `projects/components/ecc-ip/reed-solomon/rtl/fub/gf/gf_syndrome_cell.sv`. Each cell is parameterized with `ROOT_EXP = FIRST_ROOT + i` and exposes `ow_synd` (the registered value) and `ow_next` (the combinational value after the current `i_step`). The parent module concatenates the per-cell outputs into the packed `ow_synd` and `ow_synd_next` buses.

6.4.3 All-zero flag

The no-error bypass is a combinational qualifier, not a state:

```
ow_all_zero      = (ow_synd == 0)
ow_all_zero_next = (ow_synd_next == 0)
```

When `ow_all_zero_next` is true on the cycle the last symbol is stepped, the decoder core can skip the key-equation solver and Chien search and drain the block buffer unchanged.

6.4.4 Combinational next outputs

`ow_synd_next` and `ow_all_zero_next` are the values the registers would take if `i_step` is high now. A caller that knows the current beat is the last one can latch the finished syndromes immediately instead of waiting a clock for the flop update. The decoder core uses this to close timing on the re-check path (HAS chapter 3.2).

6.5 FSM policy

This block carries **no FSM** (per `vault/handbook/design/streaming-no-fsm.md`). Control is reduced to:

- `i_step`: combinational step qualifier from the caller
- `i_first`: combinational zero-and-step qualifier for the block's first beat
- `i_count`: combinational valid-symbol count for this beat
- one `m`-bit register per syndrome lane (`r_s` in `gf_syndrome_cell`)
- combinational `ow_*_next` outputs derived from the unrolled chain

Each lane is a pure multiply-accumulate datapath; the lane count and the input qualifiers replace any state machine.

6.6 Timing

- One received symbol advances every lane by one Horner step.
- Serial case (`SYMBOLS_PER_BEAT = 1`): `N_SYMBOLS` cycles per block.
- Parallel-beat case: `ceil(N_SYMBOLS / SYMBOLS_PER_BEAT)` cycles per block, with `S` chained GF constant multiplies per lane per cycle.
- The exact cycle count scales with `S` but not with `t`; the solver downstream is unaffected by `S`.

6.7 Notes

- A shortened code (`N_SYMBOLS < 2^m - 1`) is handled by treating the missing leading symbols as zero: the Horner recurrence naturally accumulates starting from `S_i = 0`, which is equivalent to zero-padding the high-order coefficients.
- **No separate clear cycle:** `i_first` zeroes the accumulator state first, then applies the first beat's symbols, so the cell is ready for the next block the cycle after the previous block ends.
- This block is an FUB. In `rs_decoder_core` it appears twice — once on the received stream and once on the corrected stream for the re-check verdict — driven by the core's own block counters and `i_step`/`i_first` qualifiers.

7 Key Equation Solver — riBM (key_equation_solver_ribm)

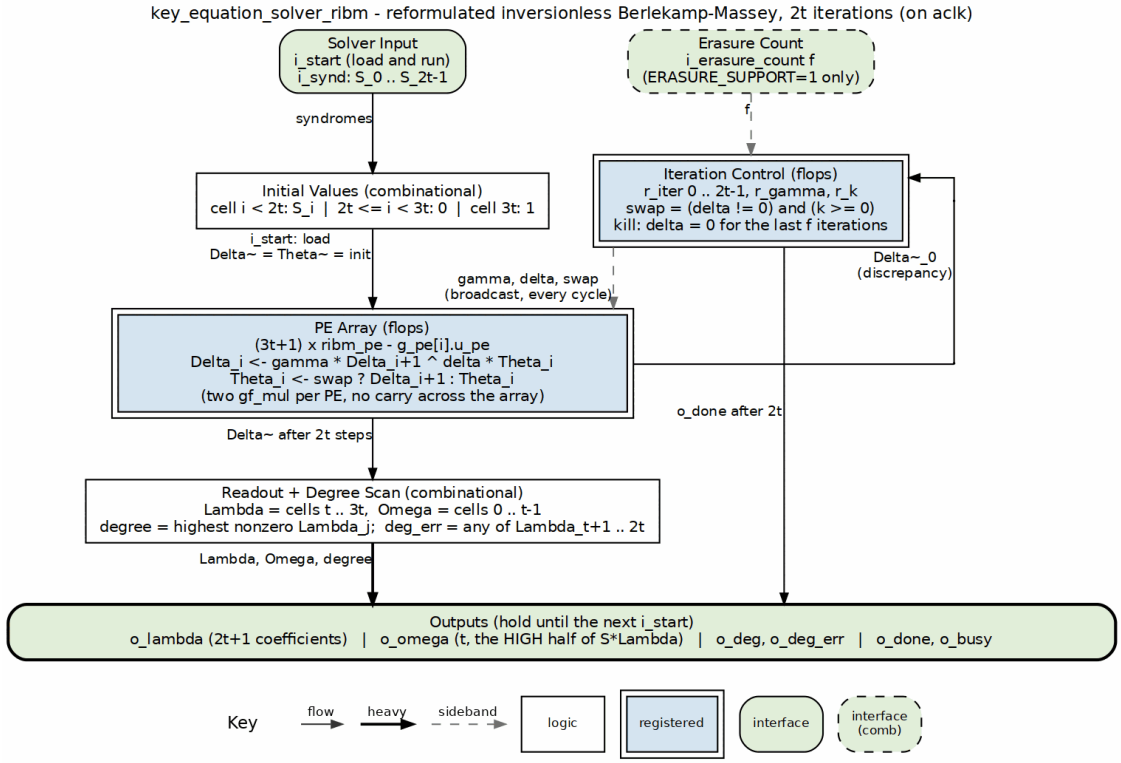
Module: key_equation_solver_ribm.sv **Location:** rtl/fub/ **Category:** datapath + iteration counter **Parent:** rs_decoder_core **Status:** landed — rtl/fub/key_equation_solver_ribm.sv, gate DV green; default KES_ALGO = "RIBM" (D11)

7.1 Purpose

`key_equation_solver_ribm` inverts the key equation from the `2t` syndromes `S_0 .. S_{2t-1}` to the error-locator polynomial `Lambda(x)` and a companion evaluator polynomial `Omega(x)` in a fixed number of cycles. It is the default key-equation solver selected by `KES_ALGO = "RIBM"` in the decoder core; the alternative is the Euclidean solver documented in chapter 2.4.

The implementation is the reformulated inversionless Berlekamp-Massey (riBM) array from Sarwate and Shanbhag, 2001. Every coefficient cell is identical, so the array is a pure datapath with a single broadcast control path. The critical path is one GF multiply and one XOR, independent of `t`.

7.1.1 Figure 2.3: riBM solver block diagram



riBM solver block diagram

7.2 Parameters

Table 2.5: riBM solver parameters

Parameter	Type	Range	Default	Meaning	PRD
SYMBOL_WIDTH	int	3..16	8	m; field is GF(2^m)	D1
PRIM_POLY	int	—	0x11D	primitive polynomial for GF(2^m)	D1
T_SYMBOLS	int	1..(2^m-2)/2	8	correctable symbol errors t	D2
ERASURE_SUPPORT	bit	0, 1	0	accept i_erasure_count and kill the last f updates	D5

7.3 Interface

Table 2.6: riBM solver ports

Signal	Direction	Width	Description
aclk	in	1	clock
aresetn	in	1	active-low synchronous reset
i_start	in	1	load syndromes and begin the 2t iterations
i_synd	in	2*T_SYMBOLS*SYMBOL_WIDTH	packed syndromes {S_{2t-1}, ..., S_0}
i_erasure_count	in	$clog2(2*T_SYMBOLS+1)	number of erasures f; only used when ERASURE_SUPPORT = 1
o_busy	out	1	solver is iterating
o_done	out	1	single-cycle pulse when the result is valid
o_lambda	out	(2*T_SYMBOLS+1)*SYMBOL_WIDTH	locator coefficients Lambda_0 .. Lambda_{2t}
o_omega	out	T_SYMBOLS*SYMBOL_WIDTH	evaluator coefficients Omega_0 .. Omega_{t-1}
o_deg	out	$clog2(2*T_SYMBOLS+1)	degree of Lambda (highest nonzero index)
o_deg_err	out	1	any of Lambda_{t+1..2t}

Signal	Direction	Width	Description
			is nonzero — more than t errors

7.4 Microarchitecture internals

The array contains `3t + 1` identical `ribm_pe` cells. Cell `i` holds the pair `Delta~_i` / `Theta~_i`. On `i_start` both registers in every cell load the same initial value:

```
i_init[i] = S_i           for i = 0 .. 2t-1
          = 0             for i = 2t .. 3t-1
          = 1             for i = 3t
```

The control state is `gamma = 1` and `k = 0`. Each of the next `2t` cycles is one iteration:

```
delta = Delta~_0
swap  = (delta != 0) && (k >= 0)
gamma <= swap ? delta : gamma
k     <= swap ? -k - 1 : k + 1
```

Each cell computes the Sarwate-Shanbhag recurrence:

```
Delta_i(r+1) = gamma(r) * Delta_{i+1}(r)  ^  delta(r) * Theta_i(r)
Theta_i(r+1) = swap ? Delta_{i+1}(r) : Theta_i(r)
```

The leaf `ribm_pe` instantiates two `gf_mul` modules and two `SYMBOL_WIDTH` registers. The `Delta_{i+1}` value comes from the neighbouring cell on the right; `gamma`, `delta`, and `swap` are broadcast to all cells. There is no feedback from the end of the array back to the beginning, so the critical path is one GF multiplier followed by one XOR at any `t`.

After exactly `2t` iterations `o_done` pulses. The outputs are combinational slices off the array:

```
o_omega = Delta~_0 .. Delta~_{t-1}
o_lambda = Delta~_t .. Delta~_{3t}
```

`o_lambda` therefore contains `2t + 1` coefficients `Lambda_0 .. Lambda_{2t}`; the top `t` coefficients are the degree-error check. The degree logic scans `Lambda_0 .. Lambda_{2t}` and raises `o_deg_err` if any coefficient above index `t` is nonzero.

With `ERASURE_SUPPORT = 1`, `i_erasure_count` carries `f` erasures. The input is the Forney syndrome list `T` zero-padded to `2t` values; the last `f` of the fixed `2t` iterations are killed by forcing `w_delta = 0` when `r_iter >= 2t - f`. The array still shifts every cycle, so the readout cells are unchanged and the result matches textbook BM on `T`.

7.5 FSM policy

There is no control FSM. The only state registers in the control path are `r_busy`, `r_iter`, `r_gamma`, and `r_k`. `r_busy` is set by `i_start` and cleared when `r_iter == 2t - 1`; `o_done` is raised on that same edge. The `swap`, `gamma`, and `k` updates are combinational or simple registered qualifiers driven by the iteration counter and the array output `Delta~_0`.

7.6 Timing

- Latency from `i_start` to `o_done` is exactly `2t` cycles.
- `o_done` is a single-cycle pulse; `o_lambda`, `o_omega`, `o_deg`, and `o_deg_err` are valid in that cycle.
- `i_start` may be asserted in the same cycle as `o_done` to begin a new block immediately.

7.7 Notes

- The evaluator produced here is the **high half** of `S(x) * Lambda(x)` (coefficients `2t .. 3t-1`), not the textbook `Omega = S*Lambda mod x^2t`. The decoder core sets the Forney evaluator's `OMEGA_HIGH_HALF` constant from `KES_ALGO`: `1` for riBM, `0` for Euclid. `dv/tbclasses/rs_model.py` is the bit-exact reference for both forms.
- The solver is bit-exact against `dv/tbclasses/rs_model.py`; the verification posture runs both solvers on the same blocks and checks identical Chien root sets and Forney values. See chapter 2.4 for the Euclidean implementation.
- Trade-off: riBM pays for a `3t + 1` cell array and a fixed `2t` cycle count in exchange for a short, t-independent critical path and no degree counters. That short path is why riBM is the default at high `t`.

8 Key Equation Solver — Euclidean (key_equation_solver_euclid)

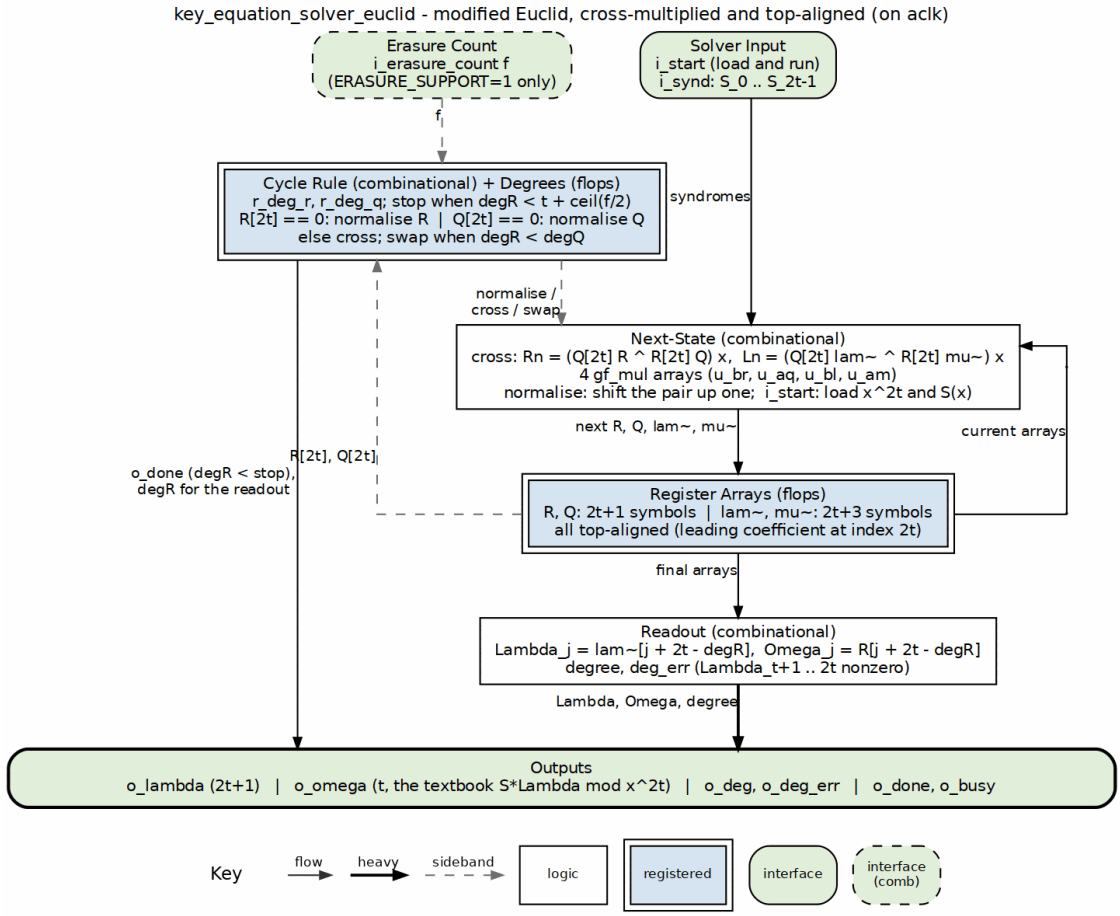
Module: key_equation_solver_euclid.sv **Location:** rtl/fub/ **Category:** datapath + cycle-rule controller **Parent:** rs_decoder_core **Status:** landed — rtl/fub/key_equation_solver_euclid.sv, gate DV green; selected by KES_ALGO = "EUCLID" (D11)

8.1 Purpose

`key_equation_solver_euclid` computes the same error-locator and error-evaluator polynomials as the riBM solver, but by the inversionless extended Euclidean algorithm. It is the alternative selected by `KES_ALGO = "EUCLID"` in the decoder core; the default is the riBM solver documented in chapter 2.3.

The implementation keeps every polynomial top-aligned, so the cross-multiply step needs no shifter. The number of cycles is data-dependent and can stop as soon as the remainder degree drops below `t`.

8.1.1 Figure 2.4: Euclid solver block diagram



Euclid solver block diagram

8.2 Parameters

Table 2.7: Euclidean solver parameters

Parameter	Type	Range	Default	Meaning	PRD
SYMBOL_WIDTH	int	3..16	8	m; field is GF(2^m)	D1
PRIM_POLY	int	—	0x11D	primitive polynomial for GF(2^m)	D1
T_SYMBOLS	int	1..(2^m-2)/2	8	correctable symbol errors t	D2
ERASURE_SUPPORT	bit	0, 1	0	accept i_erasure_count and raise the stop threshold	D5

8.3 Interface

Table 2.8: Euclidean solver ports

Signal	Direction	Width	Description
`aclk`	in	1	clock
`aresetn`	in	1	active-low synchronous reset
`i_start`	in	1	load syndromes and begin the Euclidean loop
`i_synd`	in	`2*T_SYMBOLS*SYMBOL_WIDTH`	packed syndromes `{S_{2t-1}, ..., S_0}`
`i_erasure_count`	in	`$clog2(2*T_SYMBOLS+1)`	number of erasures `f`; only used when `ERASURE_SUPPORT = 1`
`o_busy`	out	1	solver is iterating
`o_done`	out	1	single-cycle pulse when the result is valid
`o_lambda`	out	`(2*T_SYMBOLS+1)*SYMBOL_WIDTH`	locator coefficients `Lambda_0 .. Lambda_{2t}`
`o_omega`	out	`T_SYMBOLS*SYMBOL_WIDTH`	evaluator coefficients `Omega_0 .. Omega_{t-1}`
`o_deg`	out	`$clog2(2*T_SYMBOLS+1)`	degree of `Lambda` (highest nonzero index)
`o_deg_err`	out	1	any of `Lambda_{t+1..2t}`

Signal	Direction	Width	Description
			is nonzero, or the safety stop fired

8.4 Microarchitecture internals

The solver keeps four top-aligned register arrays:

```
R[0 .. 2t]          remainder, length 2t + 1
Q[0 .. 2t]          divisor,  length 2t + 1
lam~[0 .. 2t+2]     Lambda * x^(2t - degR), length 2t + 3
mu~[0 .. 2t+2]      Mu     * x^(2t - degQ), length 2t + 3
```

On `i_start`:

```
R  = x^(2t)                          (R[2t] = 1, rest 0)
Q  = S(x) top-aligned                (Q[i] = S_{i-1} for i = 1 .. 2t,
Q[0] = 0)
lam~ = 0
mu~  = x                             (mu~[1] = 1, rest 0)
degR = 2t
degQ = 2t - 1
```

Each busy cycle selects one of three actions from the leading coefficients `w_a = R[2t]` and `w_b = Q[2t]`:

```
normalise R:  R[2t] == 0            R <- R * x,  lam~ <- lam~ * x,
degR--
normalise Q:  R[2t] != 0, Q[2t] == 0  Q <- Q * x,  mu~  <- mu~  * x,
degQ--
cross:        R[2t] != 0, Q[2t] != 0
              Rn  = (Q[2t]*R  ^ R[2t]*Q)  * x
              Ln  = (Q[2t]*lam~ ^ R[2t]*mu~) * x
              if degR < degQ:
                  swap (R, lam~, degR) with (Q, mu~, degQ)
              degR = max(degR, degQ) - 1
```

The cross step is built from two independent GF multiply/XOR banks. For each `i` in `0 .. 2t`:

```
w_rn[i] = Q[2t] * R[i] ^ R[2t] * Q[i]
```

and for each `i` in `0 .. 2t+2`:

```
w_ln[i] = Q[2t] * lam~[i] ^ R[2t] * mu~[i]
```

After the cross, `R[1 .. 2t]` and `lam~[1 .. 2t+2]` load `w_rn[0 .. 2t-1]` and `w_ln[0 .. 2t+1]`, which is the `* x` shift. A normalise step does the same shift using the existing array contents.

The stop condition is:

```
finished = (degR < stop) || (degQ < 0) || (r_cycles == all-ones)
stop     = t                          when ERASURE_SUPPORT = 0
stop     = t + ceil(f / 2)            when ERASURE_SUPPORT = 1
```

To avoid spending a cycle merely noticing completion, the RTL computes the next degrees (`w_deg_r_nxt`, `w_deg_q_nxt`) combinationally and asserts `o_done` on the same clock edge as the final array write whenever `w_finished_nxt` is true.

The output stage un-shifts by `2t - degR`:

```
Lambda_j = lam~[j + 2t - degR]      for j = 0 .. 2t
Omega_j  = R[j + 2t - degR]         for j = 0 .. t-1
```

`degR` is in `0 .. t-1` for a clean stop; anything else means the safety counter retired the run and `o_deg_err` is raised. As in the riBM solver, the top `t` locator coefficients are the more-than-`t`-errors check.

`Omega` here is the textbook `S(x) * Lambda(x) mod x^(2t)`, so the decoder core sets the Forney evaluator's `OMEGA_HIGH_HALF = 0` for this solver.

8.5 FSM policy

There is no explicit state-machine enum. The active operation is selected each cycle by combinational qualifiers:

- `w_norm_r` — shift `R` / `lam~` because `R[2t] == 0`
- `w_norm_q` — shift `Q` / `mu~` because `R[2t] != 0` and `Q[2t] == 0`
- `w_cross` — perform the cross-multiply because both leading coefficients are nonzero
- `w_swap` — exchange the `(R, lam~)` and `(Q, mu~)` roles when `degR < degQ`

The controller's state is therefore held in the four register arrays plus `r_deg_r`, `r_deg_q`, `r_busy`, and the safety counter `r_cycles`. The degree comparisons are part of the datapath, which makes the per-iteration critical path longer than riBM's.

8.6 Timing

- Latency from `i_start` to `o_done` is data-dependent, between `t + 1` and `2t` iterations. The final update and `o_done` share the same clock edge.
- A zero leading syndrome at small `t` can force an extra normalise step; at `t <= 2` this produces the observed `2t + 1` cycle count.
- The safety counter `r_cycles` is `ceil(log2(4t + 8))` bits wide and stops the run before it wraps.

8.7 Notes

- The Euclidean solver is bit-exact against `dv/tbclasses/rs_model.py::euclid`. The verification posture runs both solvers on the same blocks and requires identical Chien roots and Forney values.
- Trade-off: Euclid uses shorter arrays (`2t + 1` and `2t + 3` symbols) and can stop early, but the cross-multiply feeds a degree comparison that drives the swap decision, giving a longer per-iteration critical path than riBM. That longer path is why riBM is the default and Euclid is kept as a readable cross-check.
- The actual RTL arrays give `8t + 8` GF multipliers and `8t + 8` symbol registers, not the `8t` the catalog rounds to. For the reference profile (`t = 8`) this is 72 multipliers and 72 symbol registers, compared with riBM's 50.

9 Chien Search (chien_search)

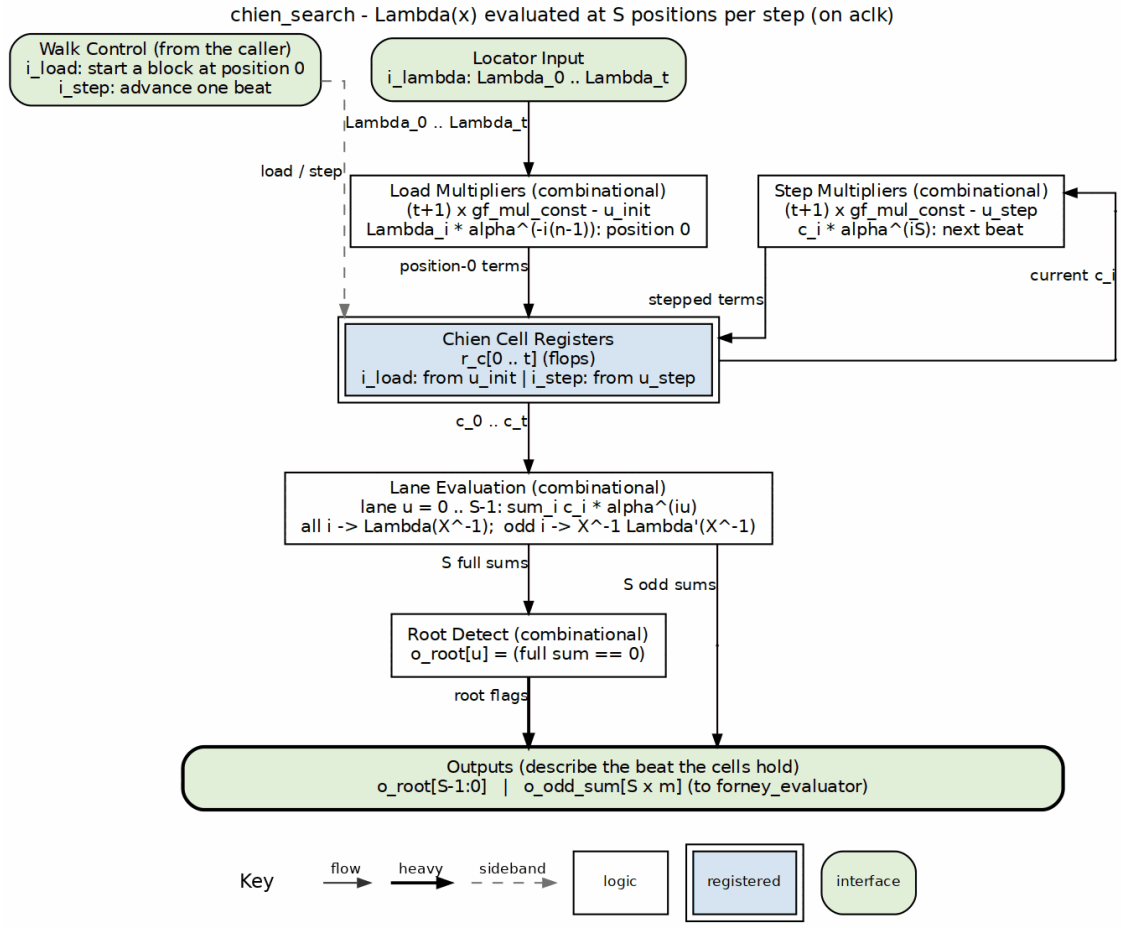
Module: chien_search.sv **Location:** rtl/fub/ **Category:** streaming datapath **Parent:** rs_decoder_core **Status:** landed — rtl/fub/chien_search.sv, gate DV green on the standing profiles

---

9.1 Purpose

`chien_search` walks the error-locator polynomial `Lambda(x)` across every symbol position of the received block. A root at position `j` means the symbol at that position is in error. Because this is a Reed-Solomon decoder, the root flag is not enough — the block also emits the odd-index partial sum `X^-1 * Lambda'(X^-1)` for each lane, which the Forney evaluator needs to compute the actual error magnitude. The math is in HAS chapter 7.3 (`../reed_solomon_has/ch07_understanding_the_math/03_decoding.md`); this page is the signal-level implementation.

9.1.1 Figure 2.5: Chien search block diagram



Chien search block diagram

9.2 Parameters

Table 2.9: Chien search parameters

Parameter	Type	Range	Default	Meaning	PRD
SYMBOL_WIDTH	int	3..16	8	m; field is GF(2^m)	D1
PRIM_POLY	int	—	'h11D	primitive polynomial	—
T_SYMBOLS	int	1..(2^m-1)/2	8	correctable symbol errors t	D2
N_SYMBOLS	int	2t+1 .. 2^m-1	2^m-1	codeword length n in symbols	D3
SYMBOLS_PER_BEAT	int	1 .. K_SYMBOLS	1	symbols evaluated per beat S	D6

9.3 Interface

Table 2.10: Chien search ports

Signal	Direction	Width	Description
aclk	in	1	one clock
aresetn	in	1	active-low synchronous reset
i_load	in	1	load Lambda and reset cells to position 0
i_lambda	in	(T_SYMBOLS + 1) * SYMBOL_WIDTH	locator coefficients Lambda_0 .. Lambda_t
i_step	in	1	advance cells to the next beat
o_root	out	SYMBOLS_PER_BEAT	root flag per lane
o_odd_sum	out	SYMBOLS_PER_BEAT * SYMBOL_WIDTH	X^-1 * Lambda'(X^-1) per lane

9.4 Microarchitecture internals

9.4.1 `t + 1` Chien cells

Symbol `j` (transmission order, `j = 0` first) has location `X_j = alpha^(n-1-j)`. It is in error when `Lambda(X_j^-1) = 0`. Cell `i` holds `Lambda_i * X_j^-i` for the beat's first position `j`:

```
load (position 0):
  r_c[i] <- Lambda_i * alpha^(-i(n-1))

step to next beat:
  r_c[i] <- r_c[i] * alpha^(iS)
```

The load constant absorbs the shortening offset: `N_SYMBOLS` is the shortened block length, so `alpha^(-i(n-1))` already points at the first transmitted position.

9.4.2 Lane structure

Lane `u` (position `j+u`) evaluates `sum_i r_c[i] * alpha^(iu)`:

```
sum[u] = sum_{i=0..t} r_c[i] * alpha^(i u)
odd[u] = sum_{i odd}     r_c[i] * alpha^(i u)

o_root[u]    = (sum[u] == 0)
o_odd_sum[u] = odd[u] = X_j^-1 * Lambda'(X_j^-1)
```

`o_root[u]` is the root flag. `o_odd_sum[u]` is the formal derivative in characteristic 2, handed to `forney_evaluator` as `i_odd_sum` so the derivative costs nothing extra.

9.4.3 Constant-multiply implementation

Every multiply in this block is by a constant. The load and step use `gf_mul_const` instances, one per cell per direction:

```
(t + 1) cells * 2 constants = 2(t + 1) gf_mul_const instances
```

The lane constants `alpha^(iu)` are built at elaboration by `build_lane_consts` and applied as `(t + 1)S` generate-loop calls to `gf_mul_fn` on a constant. Total constant multiplies: `(t + 1)(S + 2)`. For the reference profile `t = 8, S = 1` that is 27 constant multiplies. With `S > 1` the lane count scales linearly.

9.5 FSM policy

This block carries **no FSM** (per `vault/handbook/design/streaming-no-fsm.md`). The Chien walk is driven by:

- `i_load`: starts the walk at position 0
- `i_step`: moves to the next beat
- the register array `r_c[0..t]`: holds the current beat's cell values

The parent decoder core owns the position counter and the load/step sequence.

9.6 Timing

- Serial case (`SYMBOLS_PER_BEAT = 1`): `N_SYMBOLS` cycles per block to evaluate every position.
- Parallel-beat case (`SYMBOLS_PER_BEAT = S`): `ceil(N_SYMBOLS / S)` cycles per block, with `S` independent lane evaluations per cycle.
- `o_root` and `o_odd_sum` are combinational from the registered cell array, so the critical path is one lane constant multiply plus the `t + 1` input XOR tree.

9.7 Notes

- The `t + 1` coefficient order is `Lambda_0` (constant term) through `Lambda_t`. Cell `0` therefore holds `Lambda_0` unchanged and contributes the same value to every lane.
- Shortening is handled entirely by the load constants; there is no separate counter or offset register.
- The trade-off here is load-vs-step constant multiplies. Each cell carries two dedicated `gf_mul_const` instances because the initial value (position 0) and the beat-to-beat step use different constants `alpha^(-i(n-1))` and `alpha^(iS)`. A shared multiplier would need a mux and an extra cycle; the RTL pays the area to keep the walk single-cycle per beat.
- The odd-index sum is only valid when the corresponding lane has a root, but the block computes it for every lane anyway — it is the same partial sum the zero test already needs.
- Both `o_root` and `o_odd_sum` describe the beat the cells currently hold. The decoder core counts beats and pairs these outputs with the block buffer's replayed symbols. See chapter 2.6 (`forney_evaluator`) for where `o_odd_sum` goes, and chapter 2.8 (`rs_decoder_core`) for how the load/step sequence is driven.

10 Forney Evaluator (`forney_evaluator`)

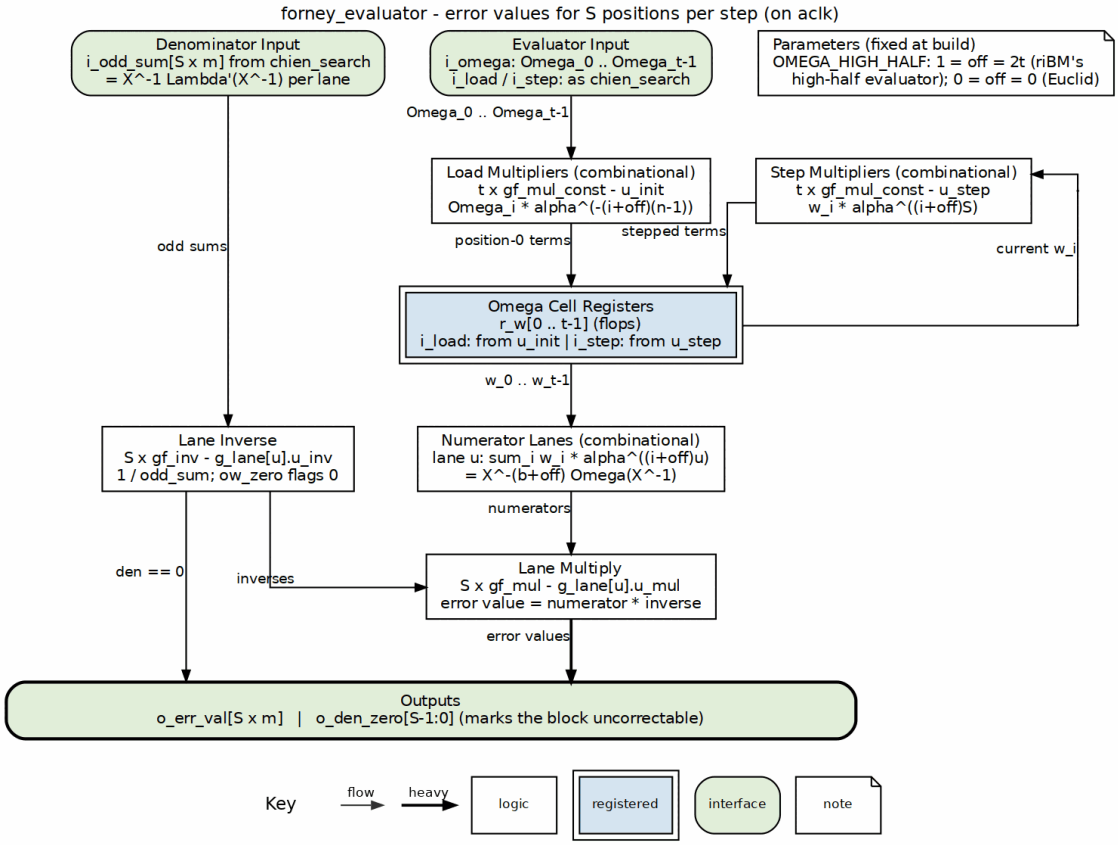
Module: `forney_evaluator.sv` **Location:** `rtl/fub/` **Category:** streaming datapath **Parent:** `rs_decoder_core` **Status:** landed — `rtl/fub/forney_evaluator.sv`, gate DV green on the standing profiles

---

10.1 Purpose

`forney_evaluator` turns a located Chien root into the actual error magnitude for that symbol. The block walks the error-evaluator polynomial `Omega(x)` in lockstep with `chien_search`, divides by the formal derivative of `Lambda(x)`, and emits one error value per lane per beat. The math is in HAS chapter 7.3 (`../reed_solomon_has/ch07_understanding_the_math/03_decoding.md`); this page is the signal-level implementation.

10.1.1 Figure 2.6: Forney evaluator block diagram



Forney evaluator block diagram

10.2 Parameters

Table 2.11: Forney evaluator parameters

Parameter	Type	Range	Default	Meaning	PRD
SYMBOL_WIDTH	int	3..16	8	m; field is GF(2^m)	D1
PRIM_POLY	int	—	'h11D	primitive polynomial	—
T_SYMBOLS	int	1..(2^m-1)/2	8	correctable symbol errors t	D2
N_SYMBOLS	int	2t+1 .. 2^m-1	2^m-1	codeword length n in symbols	D3
FIRST_ROOT	int	0..2^m-2	0	b, first consecutive root of g(x)	—
OMEGA_HIGH_HALF	bit	0/1	1	1 = riBM high-half Omega, 0 = Euclid textbook Omega	—
SYMBOLS_PER_BEAT	int	1 .. K_SYMBOLS	1	symbols corrected per beat S	D6

10.3 Interface

Table 2.12: Forney evaluator ports

Signal	Direction	Width	Description
`aclk`	in	1	one clock
`aresetn`	in	1	active-low synchronous reset
`i_load`	in	1	load `Omega` and reset cells to position 0
`i_omega`	in	`T_SYMBOLS * SYMBOL_WIDTH`	evaluator coefficients `Omega_0 .. Omega_{t-1}`
`i_step`	in	1	advance cells to the next beat
`i_odd_sum`	in	`SYMBOLS_PER_BEAT * SYMBOL_WIDTH`	`X^-1 * Lambda'(X^-1)` per lane from Chien
`o_err_val`	out	`SYMBOLS_PER_BEAT * SYMBOL_WIDTH`	error magnitude per lane
`o_den_zero`	out	`SYMBOLS_PER_BEAT`	denominator zero flag per lane

10.4 Microarchitecture internals

10.4.1 t Omega cells

The error value at position `j` is:

```
e_j = X_j^(1-b-off) * Omega(X_j^-1) / Lambda'(X_j^-1)
```

`off` depends on which solver produced `Omega`:

```
OMEGA_HIGH_HALF = 1 (riBM): off = 2t
OMEGA_HIGH_HALF = 0 (Euclid): off = 0
```

The effective exponent offset used in the RTL is:

```
OFF = b + off = FIRST_ROOT + (OMEGA_HIGH_HALF ? 2*T_SYMBOLS : 0)
```

`chien_search` already supplies `odd_sum = X_j^-1 * Lambda'(X_j^-1)`, so the formula becomes:

```
e_j = [ X_j^-(b+off) * Omega(X_j^-1) ] / odd_sum
    = [ sum_i Omega_i * X_j^-(i+OFF) ] / odd_sum
```

Cell `i` holds `Omega_i * X_j^-(i+OFF)` for the beat's first position `j`:

```
load (position 0):
  r_w[i] <- Omega_i * alpha^(-(i+OFF)(n-1))

step to next beat:
  r_w[i] <- r_w[i] * alpha^((i+OFF)S)
```

10.4.2 Lane numerator

Lane `u` (position `j+u`) sums `r_w[i] * alpha^((i+OFF)u)`:

```
num[u] = sum_{i=0..t-1} r_w[i] * alpha^((i+OFF)u)
```

These lane constants are built at elaboration by `build_lane_consts` and applied as `tS` generate-loop calls to `gf_mul_fn` on a constant, exactly like the Chien lane constants.

10.4.3 Per-lane inverse and multiply

Each lane finishes the Forney formula with one `gf_inv` and one `gf_mul`:

```
w_den_inv[u] = inv(i_odd_sum[u])
o_err_val[u] = num[u] * w_den_inv[u]
o_den_zero[u] = (i_odd_sum[u] == 0)
```

`o_den_zero[u]` cannot be true at a genuine root; when it is, the block is uncorrectable. The decoder core uses this in the final verdict.

Total primitives in the block: `2t` `gf_mul_const` instances (`t` load + `t` step), `tS` lane constants applied as `gf_mul_fn`, `S` `gf_inv`, and `S` `gf_mul`. For the reference profile `t = 8, S = 1` that is `16 + 8 + 1 + 1 = 26`. With `S > 1` the lane count scales linearly and the inverse/multiply pair is replicated per lane.

10.5 FSM policy

This block carries **no FSM** (per `vault/handbook/design/streaming-no-fsm.md`). The evaluator walk is driven by:

- `i_load`: starts the walk at position 0
- `i_step`: moves to the next beat
- the register array `r_w[0..t-1]`: holds the current beat's cell values

The parent decoder core owns the position counter and the load/step sequence, and keeps `i_step` aligned with the matching step on `chien_search`.

10.6 Timing

- Serial case (`SYMBOLS_PER_BEAT = 1`): `N_SYMBOLS` cycles per block, one error value per cycle.
- Parallel-beat case (`SYMBOLS_PER_BEAT = S`): `ceil(N_SYMBOLS / S)` cycles per block, with `S` independent lane evaluations per cycle.
- The critical path is the lane summer XOR tree feeding `gf_mul`, after the `gf_inv` lookup. The inverse is a table lookup (log / negate / antilog), not a combinational recursion, so the path stays manageable.

10.7 Notes

- `OMEGA_HIGH_HALF` is normally driven from the parent core's `KES_ALGO` parameter. The RTL does not decide which solver is correct; it exposes the offset choice and the core sets it.
- The `X^-(b+off)` factor is folded into the load and step constants, not applied as a separate post-multiplication. That keeps the per-lane path to one general multiply after the inverse.
- Shortening is handled by the load constants, just like `chien_search`: `N_SYMBOLS` is the shortened block length, so `alpha^(-(i+OFF)(n-1))` already points at the first transmitted position.
- The trade-off here is `S` inverses per beat. Every lane gets its own `gf_inv` and `gf_mul` so that all `S` positions in a beat can be corrected in the same cycle. Sharing one inverse across lanes would require either serialization or a crossbar, which would break the single-cycle-per-beat contract the decoder core relies on.
- `o_den_zero` is generated by the same `gf_inv` instance that produces the reciprocal; there is no separate zero comparator.
- The lane constants and the cell constants are all built by calling `gf_alpha_pow` at elaboration, so no runtime log/antilog tables are needed inside this block apart from the `gf_inv` tables. See chapter 2.5 (`chien_search`) for the source of `i_odd_sum`, and chapter 2.8 (`rs_decoder_core`) for how the two walks are kept aligned.

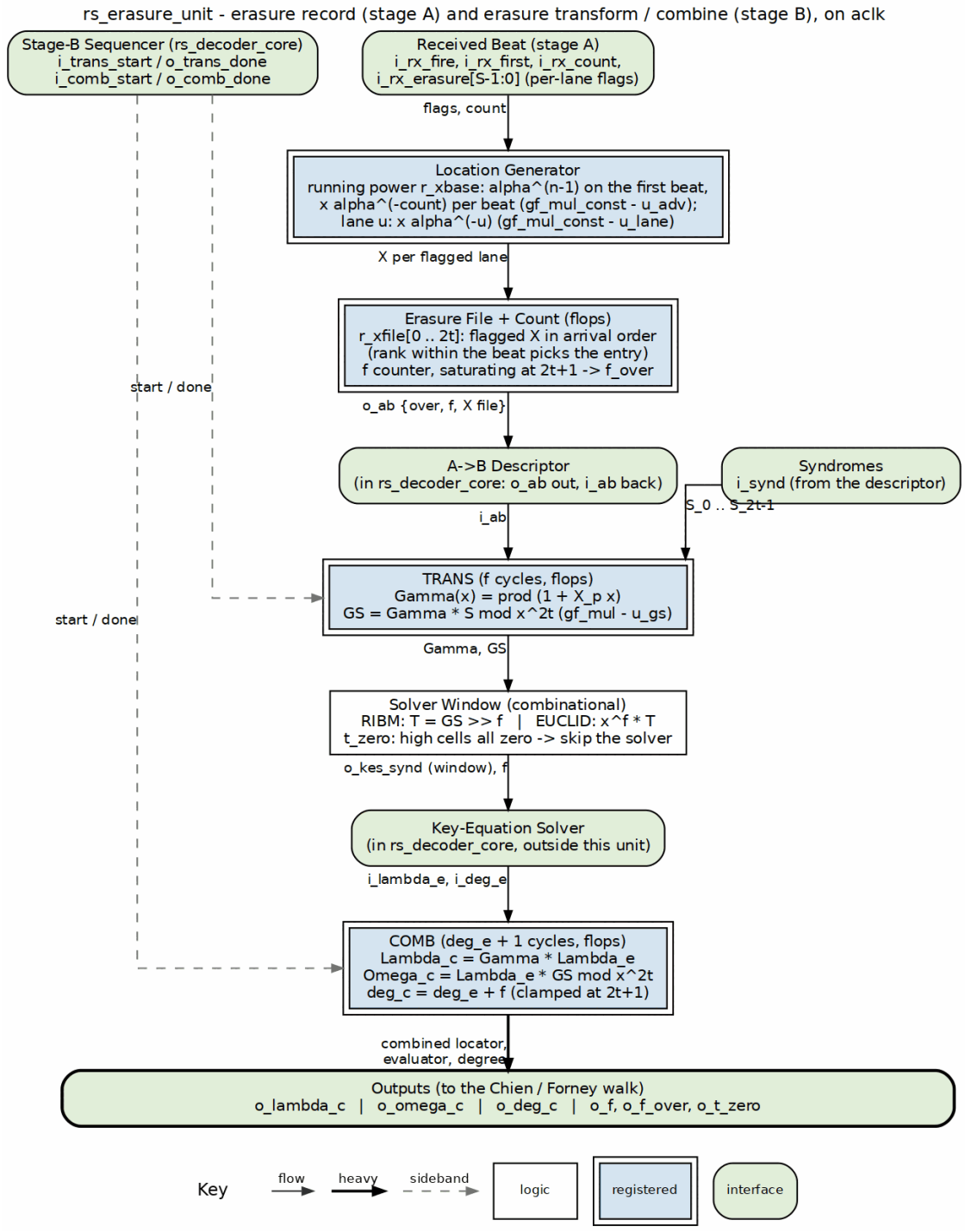
11 Erasure Unit (rs_erasure_unit)

Module: rs_erasure_unit.sv
Location: rtl/fub/
Category: erasure pre-processor (two halves joined by a packed A→B descriptor)
Parent: rs_decoder_core (instantiated only when ERASURE_SUPPORT = 1)
Status: landed — rtl/fub/rs_erasure_unit.sv, generated only under ERASURE_SUPPORT = 1 (PRD D5, default 0)

11.1 Purpose

`rs_erasure_unit` is the erasure half of the decoder. It records the positions of flagged symbols as the block arrives, turns the receive-side syndromes into the window the key-equation solver consumes, and builds the combined locator and evaluator the Chien/Forney walk consumes. The unit is split into a receive half (stage A) and a solve half (stage B), joined by a packed record that rides the decoder core's A→B descriptor. See HAS chapter 3.2 for the surrounding data flow and chapter 2.8 for how the core pipelines the unit.

11.1.1 Figure 2.7: Erasure unit block diagram



Erasure unit block diagram

11.2 Parameters

Table 2.13: Erasure unit parameters

Parameter	Type	Range	Default	Meaning	PRD
SYMBOL_WIDTH	int	3..12	8	m; field is GF(2^m)	D1
PRIM_POLY	int	primitive	'h11D	field primitive polynomial	D1
T_SYMBOLS	int	1..(2^m-2)/2	8	correctable symbol errors	D2
N_SYMBOLS	int	2t+1 .. 2^m-1	2^SYMBOL_WIDTH - 1	codeword length in symbols	D3
SYMBOLS_PER_BEAT	int	1 .. DATA_WIDTH/SYMBOL_WIDTH	1	symbols per valid/ready beat	D6
KES_ALGO	string	"RIBM", "EUCLID"	"RIBM"	solver algorithm; selects the syndrome window	D11
AB_W	int	derived	1 + clog2(2t+1) + (2t+1)*SYMBOL_WIDTH	width of the packed A→B record	—

11.3 Interface

Table 2.14: Erasure unit ports

Signal	Direction	Width	Description
`aclk`	in	1	clock
`aresetn`	in	1	active-low synchronous reset
`i_rx_fire`	in	1	a beat is accepted on the intake
`i_rx_first`	in	1	the accepted beat starts a new block
`i_rx_count`	in	`$clog2(SYMBOLS_PER_BEAT+1)`	valid symbols in the accepted beat
`i_rx_erasure`	in	`SYMBOLS_PER_BEAT`	per-lane erasure flags, valid with the beat
`o_ab`	out	`AB_W`	packed A→B record `{f_over, f, xfile[0..2t]}`
`i_ab`	in	`AB_W`	same packed record from the A→B descriptor
`i_synd`	in	`2*T_SYMBOLS*SYMBOL_WIDTH`	receive-side syndromes from the descriptor
`i_trans_start`	in	1	load and run the Gamma/transform step
`o_trans_done`	out	1	TRANS finished
`i_comb_start`	in	1	start the combine step over `Lambda_e`
`o_comb_done`	out	1	COMB finished
`i_lambda_e`	in	`(2*T_SYMBOLS+1)*SYMBOL_WIDTH`	error locator from

Signal	Direction	Width	Description
			the solver
i_deg_e	in	$clog2(2*T_SYMBOLS+1)	degree of i_lambda_e
o_kes_synd	out	2*T_SYMBOLS*SYMBOL_WIDTH	solver input syndrome window
o_f	out	$clog2(2*T_SYMBOLS+1)	erasure count f for this block
o_f_over	out	1	more than 2t erasures were flagged
o_t_zero	out	1	erasures alone explain the syndromes; solver can be skipped
o_lambda_c	out	(2*T_SYMBOLS+1)*SYMBOL_WIDTH	combined locator Gamma * Lambda_e
o_omega_c	out	2*T_SYMBOLS*SYMBOL_WIDTH	combined evaluator Lambda_e * GS mod x^2t
o_deg_c	out	$clog2(2*T_SYMBOLS+1)+1	combined degree, clamped at 2t+1

11.4 Microarchitecture internals

11.4.1 Receive half — recording the flagged locations

On every accepted beat the unit keeps a running power register holding `alpha^(n-1-pos)` for the first symbol of the beat. It is loaded with `alpha^(n-1)` on `i_rx_first` and advanced by `alpha^(-count)` each beat, so each lane's location is `X_j = alpha^(n-1-j)`. Per lane a constant multiply by `alpha^(-u)` gives the lane location without a per-symbol counter. A lane is recorded only when `i_rx_erasure[u]` is set and `u` is inside `i_rx_count` and `i_rx_fire` is high.

Flagged locations are written into a `2t+1`-entry file in arrival order. The rank of a flagged lane within its beat selects the entry, and the per-block erasure count `f` saturates at `2t+1`. The `f_over` flag is raised as soon as `f` would exceed `2t`; such a block is uncorrectable by inspection. The packed record `o_ab = {f_over, f, xfile[2t] ... xfile[0]}` is the *next* value of the file and count, because the core samples it on the block-end edge before that edge's own flags have landed in the registers.

11.4.2 Transform half — Gamma and the Forney syndrome

When the core pops the descriptor it asserts `i_trans_start`. The unit latches the unpacked record (`f` and the `xfile`) and loads the syndrome cells with `S_0..S_{2t-1}`. TRANS then walks the `f` recorded positions, updating:

```
Gamma(x) = prod_p (1 + X_p * x)
GS       = Gamma * S mod x^(2t)
```

Each cycle does one `gf_mul` per coefficient; the latency is `f` cycles (zero when `f = 0` beyond the load cycle). After TRANS the high cells of `GS` hold `T = GS >> f`, the Forney syndrome window the solver consumes.

If every high cell of `GS` is zero, the erasures alone explain every syndrome. The unit raises `o_t_zero`, which lets the core skip the solver entirely: `Lambda_e = 1`, so `Lambda_c = Gamma` and `Omega_c = GS` are already in the registers.

11.4.3 Solver input window

`KES_ALGO` selects how the `2t`-cell solver window is presented. This is the same distinction the golden model uses in `dv/tbclasses/rs_model.py`:

```
riBM:   o_kes_synd[i] = GS[i + f]  (crossbar; cells below the shift
are zero)
Euclid: o_kes_synd[i] = GS[i] when i >= f, else 0
```

riBM consumes the *dropped* window `T`; Euclid consumes the *zeroed-low* polynomial `x^f * T`. The two are equivalent because Euclid treats the polynomial as a whole, while a forward-iterating BM would be contaminated by the zero padding at the tail.

11.4.4 Combine — building the walk polynomials

After the solver returns `Lambda_e` and `deg_e`, the core asserts `i_comb_start`. COMB performs a Horner multiply over `Lambda_e`'s coefficients, high to low, producing:

```
Lambda_c = Gamma * Lambda_e
Omega_c  = Lambda_e * GS mod x^(2t)
```

Both are exactly what the Chien/Forney walk needs. The combined degree is `deg_e + f`, clamped at `2t+1`:

```
o_deg_c = r_t_zero ? f : min(deg_e + f, 2t + 1)
```

A clean solve always lands at or below `2t`.

11.4.5 Correction bound

With `f` erasures and `mu` unknown errors the decoder is bounded-distance when:

```
2 * mu + f <= 2t
```

`dv/tbclasses/rs_model.py` (`decode()` erasure path) is the bit-exact reference for this bound and for every intermediate value; the RTL was verified against it on every matrix profile.

11.4.6 Elaboration guard

The unit is instantiated only inside the decoder core generate block:

```
if (ERASURE_SUPPORT) begin : g_erasure
    rs_erasure_unit #(...) u_erasure (...);
end
```

When `ERASURE_SUPPORT = 0` every declaration and signal is tied off; the errors-only core is bit-identical to the pre-erasure version.

11.5 FSM policy

`rs_erasure_unit` has no control FSM of its own. The receive half is a set of counters and rank muxes; the solve half is two step counters (TRANS and COMB) driven by the core's `i_trans_start` and `i_comb_start` pulses. The core's solve-stage FSM (chapter 2.8) sequences those pulses and waits on `o_trans_done` and `o_comb_done`.

11.6 Timing

- Receive work: one constant multiply per lane per beat, plus the rank muxes. No extra cycle.
- `i_trans_start` to `o_trans_done`: `f` cycles when `f > 0`, one load cycle when `f = 0`.
- `i_comb_start` to `o_comb_done`: `deg_e + 1` cycles, at most `t + 1`.
- Both TRANS and COMB are one `gf_mul` deep per coefficient; the critical path is a single field multiply plus an XOR tree.

11.7 Notes

- `dv/tbclasses/rs_model.py` is the bit-exact reference. Its `decode()` erasure path uses the Forney-syndrome method (`Gamma * S mod x^2t`, drop the low `f` cells for riBM, zero the low `f` cells for Euclid), then multiplies `Gamma * Lambda_e` for the Chien/Forney walk. The RTL follows the same shape.
- The `INJ_CFG.mark` erasure-marking path is test-side only: `projects/components/utility-ip/misc/rtl/error_injector.sv` drives `out_erasure` with the beat's hit mask when `cfg_mark_erasure` is set, and `CLAUDE.md` records that this is how RAID-stripe maps or memory-controller bad-column information reach the decoder. The unit itself sees only `in_erasure` on the core port.
- With `KES_ALGO = "RIBM"` the unit shifts `GS` down by `f`; with `"EUCLID"` it zeroes the low `f` cells. Choosing the wrong window for the wrong solver is the classic erasure + BM pitfall the model caught early.

12 Decoder Core Integration (rs_decoder_core)

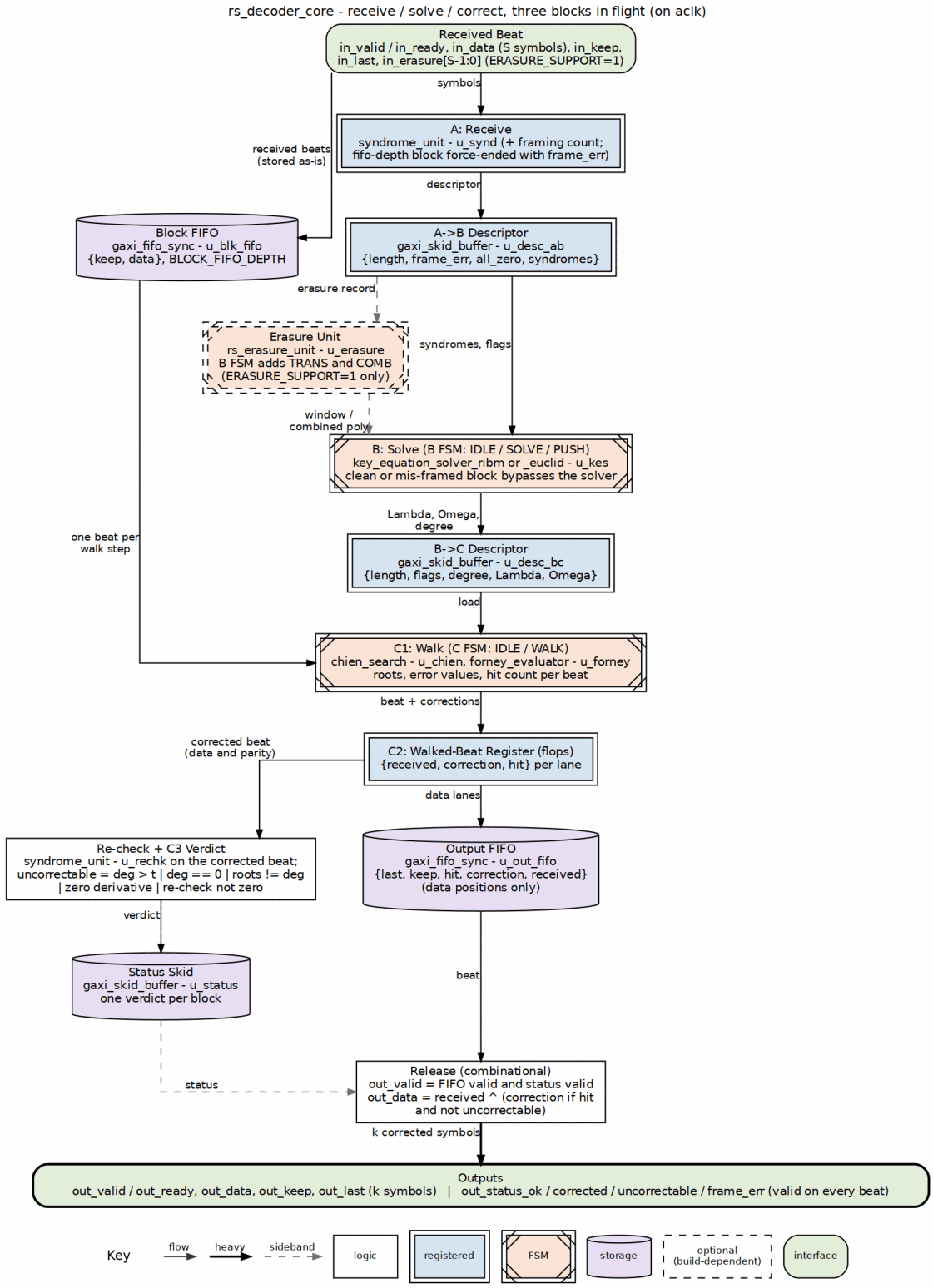
Module: rs_decoder_core.sv
Location: rtl/macro/
Category: integration block + stage control FSMs
Parent: rs_decoder / standalone decoder wrapper
Status: landed — rtl/macro/rs_decoder_core.sv, gate DV green; KES_ALG0 selects riBM (default) or Euclid at elaboration

---

12.1 Purpose

`rs_decoder_core` integrates the received-block buffer, syndrome unit, key-equation solver, Chien search, Forney evaluator, corrected-stream re-check, and output release into one valid/ready streaming block. Three stages — receive, solve, and correct — are joined by descriptor skids, so a block can be received while the previous one is solved and the one before that is corrected. The block is released only once its verdict exists, and corrections are applied at the output, not during the walk. See HAS chapter 3.2 for the overall decoder data flow.

12.1.1 Figure 2.8: Decoder core block diagram



Decoder core block diagram

12.2 Parameters

Table 2.15: Decoder core parameters

Parameter	Type	Range	Default	Meaning	PRD
SYMBOL_WIDTH	int	3..12	8	m; field is GF(2^m)	D1
PRIM_POLY	int	primitive	'h11D	field primitive polynomial	D1
T_SYMBOLS	int	1..(2^m-2)/2	8	correctable symbol errors	D2
N_SYMBOLS	int	2t+1 .. 2^m-1	2^SYMBOL_WIDTH - 1	codeword length in symbols	D3
FIRST_ROOT	int	0 .. 2^m-2	0	first consecutive root b of the generator	D1
DATA_WIDTH	int	SYMBOL_WIDTH ..	SYMBOL_WIDTH	bus width; must be a multiple of SYMBOL_WIDTH	D9
SKID_DEPTH	int	2..8	2	depth of descriptor skid buffers	D9
BLOCK_FIFO_DEPTH	int	power of two	2^ceil(log2((n+2t)/S+8))	block buffer depth in beats	HAS 5.2
KES_ALGO	string	"RIBM", "EUCLID"	"RIBM"	key-equation solver	D11

Parameter	Type	Range	Default	Meaning	PRD
				algorithm	
ERASURE_SUPPORT	bit	0, 1	0	per-lane erasure flags and erasure decode	D5
K_SYMBOLS	int	derived	N_SYMBOLS - 2*T_SYMBOLS	data symbols per block	—
SYMBOLS_PER_BEAT	int	derived	DATA_WIDTH / SYMBOL_WIDTH	symbols per beat S	—
STATUS_CNT_WIDTH	int	derived	clog2(2t+1) if erasures else clog2(t+1)	width of out_status_corrected	—

12.3 Interface

Table 2.16: Decoder core ports

Signal	Direction	Width	Description
`aclk`	in	1	clock
`aresetn`	in	1	active-low synchronous reset
`in_valid` / `in_ready`	in / out	1	valid/ready for received beats
`in_data`	in	`DATA_WIDTH`	received symbols, lane 0 in the low bits
`in_keep`	in	`SYMBOLS_PER_BEAT`	present-symbol mask; low-aligned, partial only on a block's last beat
`in_last`	in	1	this beat carries the block's last received symbol
`in_erasure`	in	`SYMBOLS_PER_BEAT`	per-lane erasure flags; dead unless `ERASURE_SUPPORT = 1`
`out_valid` / `out_ready`	out / in	1	valid/ready for corrected data beats
`out_data`	out	`DATA_WIDTH`	corrected data symbols; parity lanes dropped
`out_keep`	out	`SYMBOLS_PER_BEAT`	present-symbol mask for the data beats
`out_last`	out	1	this beat carries the block's k-th

Signal	Direction	Width	Description
			data symbol
out_status_ok	out	1	block had no errors
out_status_corrected	out	STATUS_CNT_WIDTH	number of symbols corrected (0 .. 2t with erasures, 0 .. t without)
out_status_uncorrectable	out	1	correction failed; block passes through unchanged
out_status_frame_err	out	1	block length was not N_SYMBOLS or a partial beat preceded the last

12.4 Microarchitecture internals

12.4.1 Stage A — receive

Every accepted beat is written to the block FIFO as `{in_keep, in_data}` and fed to the receive `syndrome_unit`. On the block's last symbol (`in_last`) a descriptor is pushed to stage B. The descriptor carries length, a frame-error flag, an all-zero-syndrome flag, and the syndromes; when `ERASURE_SUPPORT = 1` it also carries the erasure unit's packed A→B record. A block that reaches the FIFO depth without seeing `in_last` is force-ended there with `frame_err`, so a runaway input stream cannot deadlock the core. The next `in_last` resynchronizes the receiver.

A partial beat on any beat other than the last is also flagged as a framing error, because `in_keep` is only allowed to be partial on the block's final beat.

12.4.2 Stage B — solve

A clean block (`all_zero`) or a mis-framed block bypasses the solver. Otherwise the selected key-equation solver runs on the syndrome window from the descriptor (or from the erasure unit when erasures are enabled). When the solver finishes, `{Lambda, Omega, degree}` join a new B→C descriptor. With erasures the descriptor carries the *combined* locator and evaluator from `rs_erasure_unit` and the combined degree, which can reach `2t`.

The B stage is the only place in the core that holds a control FSM; the solver itself has its own FSM on its own page (chapter 2.3 for riBM, chapter 2.4 for Euclid). The two generate branches are:

```
if (!ERASURE_SUPPORT) begin : g_b_err
    typedef enum logic [1:0] {B_IDLE, B_SOLVE, B_PUSH} b_state_t;
    ...
end else begin : g_b_erasure
    typedef enum logic [2:0] {BE_IDLE, BE_TRANS, BE_SOLVE, BE_COMB,
BE_PUSH} be_state_t;
    ...
end
```

12.4.3 Stage C — correct

Stage C is split into three pipeline phases to close timing at 100 MHz on Artix-7:

- **C1** loads the Chien/Forney cells and walks the block one beat per cycle, reading the block FIFO. Each cycle produces root flags, odd sums, Forney values, and per-lane hit flags.
- **C2** captures that result into a pipeline register. From that register the corrected symbol stream is fed to a second `syndrome_unit` (the re-check), and the received symbols together with their corrections and hit flags are pushed into the output FIFO.

- **C3** computes the verdict one cycle later, reading the re-check's *registered* `all_zero` output. That extra cycle ends the S-fold Horner chain at the syndrome cells' own flops instead of carrying it into the status path; it costs one cycle per block.

The verdict is final on the last beat:

```
uncorrectable = correct && ( bad
                           || roots != degree
                           || den_zero_hit
                           || !rechk_zero )
```

where `bad` already covers `deg > t`, `deg == 0` (errors-only), and solver degree-error. A block is released only with its verdict; if the verdict is uncorrectable the received data pass through unchanged.

12.4.4 Descriptor records

Table 2.17: Descriptor record layouts (`DAB_W` and `DBC_W`)

Descriptor	Field	Width	Source / meaning
A→B	`len`	`CNT_W = clog2(BFD*S+1)`	number of received symbols in the block
A→B	`frame_err`	1	length != `n`, partial beat before last, or FIFO force-end
A→B	`all_zero`	1	receive syndromes all zero
A→B	`syndromes`	`2t*M`	receive-side syndrome vector `S_0..S_{2t-1}`
A→B	`erasure record`	`ERAB_W`	`{f_over, f, xfile[0..2t]}`; present only if `ERASURE_SUPPORT = 1`
B→C	`len`	`CNT_W`	number of received symbols in the block
B→C	`frame_err`	1	passed from A→B

Descriptor	Field	Width	Source / meaning
B→C	`all_zero`	1	passed from A→B
B→C	`correct`	1	block will be corrected (not bypassed)
B→C	`bad`	1	pre-computed solver failure or `f_over`
B→C	`deg`	`DEGC_W`	locator degree; up to `2t` with erasures
B→C	`lambda`	`LAM_N*M`	locator coefficients
B→C	`omega`	`OM_N*M`	evaluator coefficients

`LAM_N` is `2t+1` with erasures, `t+1` without. `OM_N` is `2t` with erasures, `t` without. `DEGC_W` is `clog2(2t+1)+1` with erasures, `clog2(2t+1)` without.

12.4.5 FIFO sizing

Table 2.18: Decoder core FIFO sizing

FIFO	Depth	Width	Contents
Block FIFO	`2^ceil(log2((n+2t)/S+8))` beats	`S + S*M`	`{keep[S], received[S*M]}`
Output FIFO	`2^ceil(log2(2*ceil(k/S)+8))` beats	`1 + 2S + 2S*M`	`{last_data_beat, keep_data[S], hit[S], correction[S*M], received[S*M]}`

The block FIFO default depth is the first power of two at or above `(n + 2t)/S + 8`. The output FIFO depth is the first power of two at or above `2*ceil(k/S) + 8`.

12.4.6 Status encodings

Table 2.19: Decoder core status outputs

Signal	Meaning
`out_status_ok`	`all_zero` was true and no frame error occurred

Signal	Meaning
`out_status_corrected`	Number of corrected symbols when `correct = 1` and the block is not uncorrectable; saturates at the width's maximum
`out_status_uncorrectable`	Solver failure, root/degree mismatch, zero derivative at a root, or re-check syndromes non-zero
`out_status_frame_err`	Block framing violated; block passes through uncorrected

All four status signals are valid on every beat of the released block; a consumer may sample them with `out_last`.

12.5 FSM policy

The decoder core contains two minimal stage FSMs; every algorithmic block inside a stage is FSM-free.

- **B solve FSM** sequences the solver and, when erasures are enabled, the erasure unit's TRANS and COMB steps. Errors-only states: `B_IDLE`, `B_SOLVE`, `B_PUSH`. Erasure states: `BE_IDLE`, `BE_TRANS`, `BE_SOLVE`, `BE_COMB`, `BE_PUSH`. `B_PUSH` / `BE_PUSH` exist only to cover the backpressured case where the next descriptor skid is not ready on the cycle the result is produced.
- **C walk FSM** has two states: `C_IDLE` and `C_WALK`. The load into `C_WALK` can happen straight off the previous block's last step, so block boundaries do not add a dead cycle.

The descriptor skid buffers (`gaxi_skid_buffer`) decouple the stages and absorb backpressure. The Euclid solver's internal control is documented in chapter 2.4, not here.

12.6 Timing

- Receive phase: `ceil(N_SYMBOLS / SYMBOLS_PER_BEAT)` beats.
- Solve phase:
  - Bypass: one cycle.
  - Errors-only: `2t` solver iterations (riBM) or `t+1` to `2t+1` cycles (Euclid can stop early).
  - With erasures: `f` TRANS cycles + solver iterations + `deg_e + 1` COMB cycles, plus a bubble per block boundary because the next block's TRANS cannot start until the current one pushes.
- Correct phase:
  - Walk: `ceil(N_SYMBOLS / SYMBOLS_PER_BEAT)` beats.
  - Verdict: one additional cycle after the last beat leaves C2.
- Measured sustained throughput: `N_SYMBOLS/SYMBOLS_PER_BEAT + 1` cycles per block at `S = 4` and `S = 8` for the reference profiles.

The extra verdict cycle is the price of registering the re-check result. It closed the 100 MHz timing path on Artix-7 that the unregistered S-fold Horner chain could not meet.

12.7 Notes

- `dv/tbclasses/rs_model.py` is the bit-exact reference for the whole decoder. The model's `decode()` method, with `erasures` set, was used to validate the erasure path against reedsolo on every profile.
- A mis-framed block has its first `length - 2t` symbols emitted as data; when `length <= 2t` all symbols are emitted as data. The block is not corrected.
- `KES_ALGO` selects only the solver and the Forney evaluator-form constant; Chien, the buffer, the corrector, and the re-check are identical for both algorithms.
- `ERASURE_SUPPORT = 0` removes the erasure unit, ties off `in_erasure`, and keeps the errors-only core bit-for-bit. `ERASURE_SUPPORT = 1` widens the status counter to `clog2(2t+1)` and changes the B-stage occupancy; see chapter 2.7 for the unit itself.
- The output FIFO is sized to absorb the k/S data beats plus margin; the status skid holds one entry per block. A block leaves only when both the output FIFO entry and the status entry are valid, which is why an uncorrectable block still leaves exactly as it arrived.

13 Core Signals

This page presents the valid/ready core signals as a contract table rather than restating the prose interface description in `../reed_solomon_has/ch04_interfaces/01_core_interface.md`. The direction, reset value, and creator block for each signal are the source of truth here.

13.1 Reset strategy

All core logic uses **synchronous active-low reset** (`aresetn`), the house clocking convention. Reset clears block state, counters, and control flags; the data-carrying arrays themselves are not reset. In `rs_encoder_core.sv` the reset block clears `r_drain`, `r_count`, `r_drain_left`, and `frame_err` (`rs_encoder_core.sv:191-195`); the LFSR parity registers and the output skid buffer contents are not reset, and their pointers make the old contents logically invalid until overwritten. In `rs_decoder_core.sv` reset clears the stage-A receive counters, the descriptor skid and FIFO control flags, the stage-B solver handshake state, and the stage-C walk/output/status pipeline (`rs_decoder_core.sv:279-290`, `461-484`, `538-566`, `735-754`, `799-843`, `849-862`); the block FIFO and output FIFO data arrays are not reset.

13.2 rs_encoder_core signal contract

Table 3.1: rs_encoder_core signal contract

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
aclk	in	1	—	clock source	one clock for the whole core	—
aresetn	in	1	—	reset source	synchronous active-low reset	—
in_valid	in	1	—	upstream	stable until in_ready high	encoder core
in_ready	out	1	0	encoder core	combinational ! r_drain && w_skid_wr_ready; low during parity drain	encoder core
in_data	in	DATA_WIDTH	—	upstream	data symbols, low-aligned; partial final beat qualified by in_keep	encoder core
in_keep	in	SYMBOLS_PER_BEAT	—	upstream	present-symbol	encoder core

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
					mask, low-aligned; partial only on final beat	
`in_last`	in	1	—	upstream	this beat carries the k-th data symbol	encoder core
`out_valid`	out	1	0	encoder core	registered skid output; data beats then parity beats	encoder core
`out_ready`	in	1	—	downstream	may depend on `out_valid`	encoder core
`out_data`	out	`DATA_WIDTH`	0	encoder core	data while `!r_drain`, parity while `r_drain`	encoder core
`out_keep`	out	`SYMBOLS_PER_BEAT`	0	encoder core	full except on final data and final parity	encoder core

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
					beats	
`out_last`	out	1	0	encoder core	this beat carries the n-th coded symbol	encoder core
`frame_err`	out	1	0	encoder core	pulse when `in_last` does not coincide with k data symbols	encoder core

13.3 rs_decoder_core signal contract

Table 3.2: rs_decoder_core signal contract

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
in_valid	in	1	—	upstream	stable until in_ready high	decoder core
in_ready	out	1	0	decoder core	high while block FIFO and A->B descriptor skid have space	decoder core
in_data	in	DATA_WIDTH	—	upstream	received symbols, low-aligned	decoder core
in_keep	in	SYMBOLS_PER_BEAT	—	upstream	present-symbol mask; partial only on final beat	decoder core
in_last	in	1	—	upstream	this beat carries the n-th received symbol	decoder core
in_erasure	in	SYMBOLS_PER_BEAT	—	upstream	per-lane known-bad	erasure unit, decoder

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
					symbol flags; dead unless ERASURE_SUPPORT	core
out_valid	out	1	0	decoder core	high once the block verdict is final	decoder core
out_ready	in	1	—	downstream	may depend on out_valid	decoder core
out_data	out	DATA_WIDTH	0	decoder core	corrected data symbols; parity lanes dropped	decoder core
out_keep	out	SYMBOLS_PER_BEAT	0	decoder core	present-symbol mask; partial only on final beat, data lanes only	decoder core
out_last	out	1	0	decoder core	this beat carries the k-th data symbol	decoder core

Signal	Dir	Width	Reset value	Creates / consumes	Contract	Source block page
out_status_ok	out	1	0	decoder core	held for every beat: block had no errors	syndrome unit, decoder core
out_status_corrected	out	STATUS_CNT_WIDTH	0	decoder core	held for every beat: symbols corrected (0 .. t, or 0 .. 2t with erasures)	Chien search, Forney evaluator, decoder core
out_status_uncorrectable	out	1	0	decoder core	held for every beat: correction failed; data pass through unchanged	Chien search, Forney evaluator, decoder core
out_status_frame_err	out	1	0	decoder core	held for every beat: block length was not n	decoder core

STATUS_CNT_WIDTH is ERASURE_SUPPORT ? $clog2(2*T_SYMBOLS+1) : $clog2(T_SYMBOLS+1) (rs_decoder_core.sv:133-134).

13.4 Timing rules

- `in_last` is the sole block boundary; the cores do not infer boundaries by counting.
- `out_status_*` describe the block being emitted and are valid on every beat of it, first to last. A consumer may sample them with `out_last` or with the first beat.
- An uncorrectable block leaves exactly as received; the corrector is disabled by `out_status_uncorrectable` at the output mux (`rs_decoder_core.sv:931-933`).

---

14 Signal Contracts and Golden Model

14.1 What a contract is

A signal contract in this repo is a machine-checkable statement of what a signal is allowed to do. Each contract has three parts, in order:

1. **Term list:** every signal or expression in play, with its defining expression and `file:line` citation.
2. **Invariants:** the strict relationships between terms, each cited, each stating which rows of the decision table it renders impossible.
3. **Decision table:** one row per combination of the terms. Rows the invariants forbid are marked `ILLEGAL`; legal rows carry the resulting output.

The canonical methodology is in `vault/handbook/design/signal-contracts-and-kmaps.md`; the shared generator machinery is in `bin/kmaps/`.

14.2 Which signals carry the contracts

The landed RTL concentrates the non-obvious control and qualifier decisions in the decoder core, the solver interface, and the erasure unit. The contracts for these areas are cited directly against the `.sv` source today.

Table 4.1: Contract areas in the landed RTL

Contract area	Signals	RTL citation
Stage A boundary / frame error	`w_in_count`, `w_force_end`, `w_block_end`, `w_len`, `w_frame_err`	`rs_decoder_core.sv:228-232`
A->B descriptor valid	`w_dab_wr_valid`, `w_dab_wr_data`	`rs_decoder_core.sv:271-276`
Syndrome all-zero / frame-error bypass	`w_b_all_zero`, `w_b_frame_err`, `w_b_bypass`	`rs_decoder_core.sv:322-324, 421`
Erasure mux into solver	`w_kes_synd`	`rs_decoder_core.sv:338-339`
Solver start and handshake	`w_kes_start`, `w_kes_done`, `w_kes_busy`, `w_kes_deg_err`	`rs_decoder_core.sv:330, 443-446`; `key_equation_solver_ribm.sv:84-89`; `key_equation_solver_euclid.sv:94-99`
Solve-bypass bad verdict	`w_b_bad` (errors-only), `w_b_bad_final` (erasures)	`rs_decoder_core.sv:432, 498-506`
B->C descriptor valid	`w_dbc_wr_valid`, `w_dbc_wr_data`	`rs_decoder_core.sv:449-458, 514-535`
Final uncorrectable verdict	`w_uncorrectable_final`	`rs_decoder_core.sv:624, 790-794`
Status hold / output pack	`r_st_data`, `out_status_ok`, `out_status_corrected`, `out_status_uncorrectable`, `out_status_frame_err`	`rs_decoder_core.sv:658, 836-839, 926-927`
Erasure unit transforms	`o_trans_done`,	`rs_erasure_unit.sv:10`

Contract area	Signals	RTL citation
	o_comb_done, o_f_over, o_t_zero, o_kes_synd, o_lambda_c, o_omega_c, o_deg_c	4-117, 245-247

14.3 Citation posture

The RTL has already landed, so the contracts cite `.sv` file and line today; no re-pointing is needed. There is no generated workbook for RS yet (unlike BCH's `bch_signal_contracts.xlsx` produced by `projects/components/ecc-ip/bch/docs/gen_bch_signal_contracts_kmaps.py`). The natural follow-up is to clone that generator: the expressions are concentrated and cited in Table 4.1, the shared machinery lives in `bin/kmaps/`, and the BCH generator is the working pattern to adapt.

14.4 The golden-model contract

`dv/tbclasses/rs_model.py` is the bit-exact reference for every decoder block. It mirrors the RTL algorithms step for step (Horner syndromes in transmission order, the riBM key-equation solver, Chien search from the first transmitted position, Forney evaluation with first root `b`), and it is validated against the `reedsolo` package before any RTL is scored against it. The area rule is explicit: field arithmetic comes from `reedsolo` or `galois`, never a hand-rolled GF table in a test (`CLAUDE.md`, `README.md`).

Two additional equivalence checks are part of the contract:

- **Dual-solver equivalence.** The decoder core is built with `KES_ALGO` selectable between `"RIBM"` and `"EUCLID"`. The Nexys A7 loop harness runs both solvers on the same injected stream and compares their Chien roots and Forney values per block; the measured result is bit-identical corrected output and identical verdicts across the error range (`CLAUDE.md`).
- **Erasures path.** With `ERASURE_SUPPORT = 1` the error injector can mark per-lane erasures through `INJ_CFG.mark` (PRD D5, TASK-002). The erasure unit transforms the syndromes, drives the solver window, and combines the erasure locator with the error locator so the Chien/Forney walk operates on the combined polynomial (`rs_decoder_core.sv:362-408`, `rs_erasure_unit.sv`).

14.5 Where formal verification stands

No formal verification directory or `.sby` files exist under `projects/components/ecc-ip/reed-solomon/` today. Coverage is by directed and randomized cocotb tests against `rs_model.py`; formal contracts and SVA assertions have not been added yet.